

04_Final_Project_cs-bb.executed

October 10, 2026

1 90819-Python Final Project: Effect of Proximity to Polluters on Home Values and Vacancy Rates

Pittsburgh consistently ranks as one of the worst polluted areas in the United States. Allegheny county in particular, ranks as one of the worst, ranking 23 in the US in 2026 [American Lung Association \(2026\)](#). Industrial polluters can cause major adverse health effects due to the release of harmful particles into the air, such as benzene, lead and lead compounds, mercury and mercury compounds, methanol, and nitrates. We want to see if proximity to industrial polluters have any affect on where people choose to live. We proxy this by home values and vacancy rates.

Our primary research question is as follows: ##### *How does proximity to major industrial polluters shape home values, vacancy, and housing distress in Mon Valley municipalities, compared with the rest of Allegheny County?*

1.1 Analysis Organization

We organize our analysis into the following key sections: 1. Data Processing - Housing and demographic data sources: The US ACS 2020-2024 5-year survey data at the census tract level, and Allegheny County Property Assessments data at the parcel level, and Allegheny County municipalities boundaries. - Pollution data: EPA Toxic Release Inventory (TRI) data, and EPA Risk Screening Environmental Indicators (RSEI) data. - Exploratory data analysis: Bar charts, histograms, density plots, and maps exploring the data for trends. 3. Regression Analysis - Modeling the effect of proximity to industrial polluters on property sale prices for residential properties. - Modeling the effect proximity of industrial polluters on vacancy rates. - Each model includes an OLS regression and a logit regression for properties located in Mon Valley. 4. Conclusions, Limitations, and Areas for Further Study

1.2 1. Data processing

The data processing for this analysis requires running scripts 01-03, which handle the loading, cleaning, and merging of the various datasets used in the study. In this document, we provide an overview of the steps involved and the structure of the resulting datasets, but the full processing code is contained within those scripts. The output of these scripts is a cleaned and merged dataset ready for analysis, which, due to size constraints, are stored on Google Drive.

The dataframe after all of the data are cleaned and merged contains 585,005 rows and 52 columns. The analytical dataset is available as a zip file in our github repo as `90819_analysis_data.zip`. Please download and save this file before running this main analysis file.

The subset used for final analysis and regression contains 24,139 rows and 52 columns. The regressions will be run on this final dataset.

1.2.1 Import Libraries and Load Libraries

```
[1]: # note, you may need to install these libraries first
import pandas as pd
import geopandas as gpd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import pytidycensus as tc
# import mapclassify as mc
from pathlib import Path
from patsy import dmatrices
#from sklearn.linear_model import LinearRegression
import pyfixest as pf
import statsmodels.api as sm
from pathlib import Path
```

```
C:\Users\Bryce\AppData\Local\Temp\ipykernel_22388\1226118530.py:7: UserWarning:
Mapping functions unavailable due to import error: ModuleNotFoundError. To use
mapping features, ensure all dependencies are properly installed: pip install
pytidycensus[map]
import pytidycensus as tc
```

1.3 1.1. Parcel data

The housing and demographic data includes information from the US ACS 2020-2024 5-year survey at the census tract level, Allegheny County Property Assessments data at the parcel level, and Allegheny County municipalities boundaries. We use this data to understand the local distribution of population and demographic characteristics, housing stock, and property values, which we then use to analyze the effect of proximity to industrial polluters on home values and vacancy rates.

The code below demonstrates how we loaded, cleaned, and merged the housing and demographic data for our analysis. The full code used to produce these datasets is available in script 01_parcel_analysis.ipynb and acs_pull_code.ipynb.

```
[2]: output_dir = Path(r"C:\Users\Bryce\Main\CMU\Fall 2026\Python II\Final Project")
# load full dataframe
property_gdf = gpd.read_file(output_dir / "90819_analysis_data_full.geojson")

# load final analysis dataset
analysis_gdf = gpd.read_file(output_dir / "90819_analysis_data.geojson")
```

```
[3]: # create dummy variable for Mon Valley Municipalities
monvalley_municipalities = [
    "FORWARD",
    "ELIZABETH",
```

```

"WEST ELIZABETH",
"JEFFERSON HILLS",
"CLAIRTON",
"LINCOLN",
"LIBERTY",
"GLASSPORT",
"PORT VUE",
"MCKEESPORT",
"WEST MIFFLIN",
"DRAVOSBURG",
"DUQUENSNE",
"NORTH VERSAILLES",
"EAST PITTSBURGH",
"NORTH BRADDOCK",
"BRADDOCK",
"WHITAKER",
"MUNHALL",
"WEST HOMESTEAD",
"HOMESTEAD",
"RANKIN",
"SWISSVALE"
]

property_gdf["mon_valley"] = property_gdf["name"].isin(monvalley_municipalities)
analysis_gdf["mon_valley"] = analysis_gdf["name"].isin(monvalley_municipalities)

```

We will start by looking at the property assessor data.

```

[4]: # get number of buildings by class
buildings_by_class = (property_gdf
                        .groupby("classdesc")
                        .size()
                        .reset_index(name="count")
                        .sort_values(by="count", ascending=False)
                        )

buildings_by_class.plot.bar(x="classdesc", y="count", title="Number of Parcels_
↳by Class")

print(f"Top 3 parcel uses by class: \n1. {buildings_by_class.
↳iloc[0]['classdesc']} ({buildings_by_class.iloc[0]['count']})\n2.
↳{buildings_by_class.iloc[1]['classdesc']} ({buildings_by_class.
↳iloc[1]['count']})\n3. {buildings_by_class.iloc[2]['classdesc']}
↳({buildings_by_class.iloc[2]['count']})")

# filter to the year 2024 and check again
assessment_parcel_2024_df = property_gdf[property_gdf["sale_year"]==2024]

```

```

buildings_by_class_2024 = (assessment_parcel_2024_df
                            .groupby("classdesc")
                            .size()
                            .reset_index(name="count")
                            .sort_values(by="count", ascending=False))

print(f"Top 3 parcel uses by class in 2024: \n1. {buildings_by_class_2024.
    ↳iloc[0]['classdesc']} ({buildings_by_class_2024.iloc[0]['count']})\n2.↳
    ↳{buildings_by_class_2024.iloc[1]['classdesc']} ({buildings_by_class_2024.
    ↳iloc[1]['count']})\n3. {buildings_by_class_2024.iloc[2]['classdesc']}]↳
    ↳({buildings_by_class_2024.iloc[2]['count']})")
# print("We want to focus on residential properties for this analysis.")

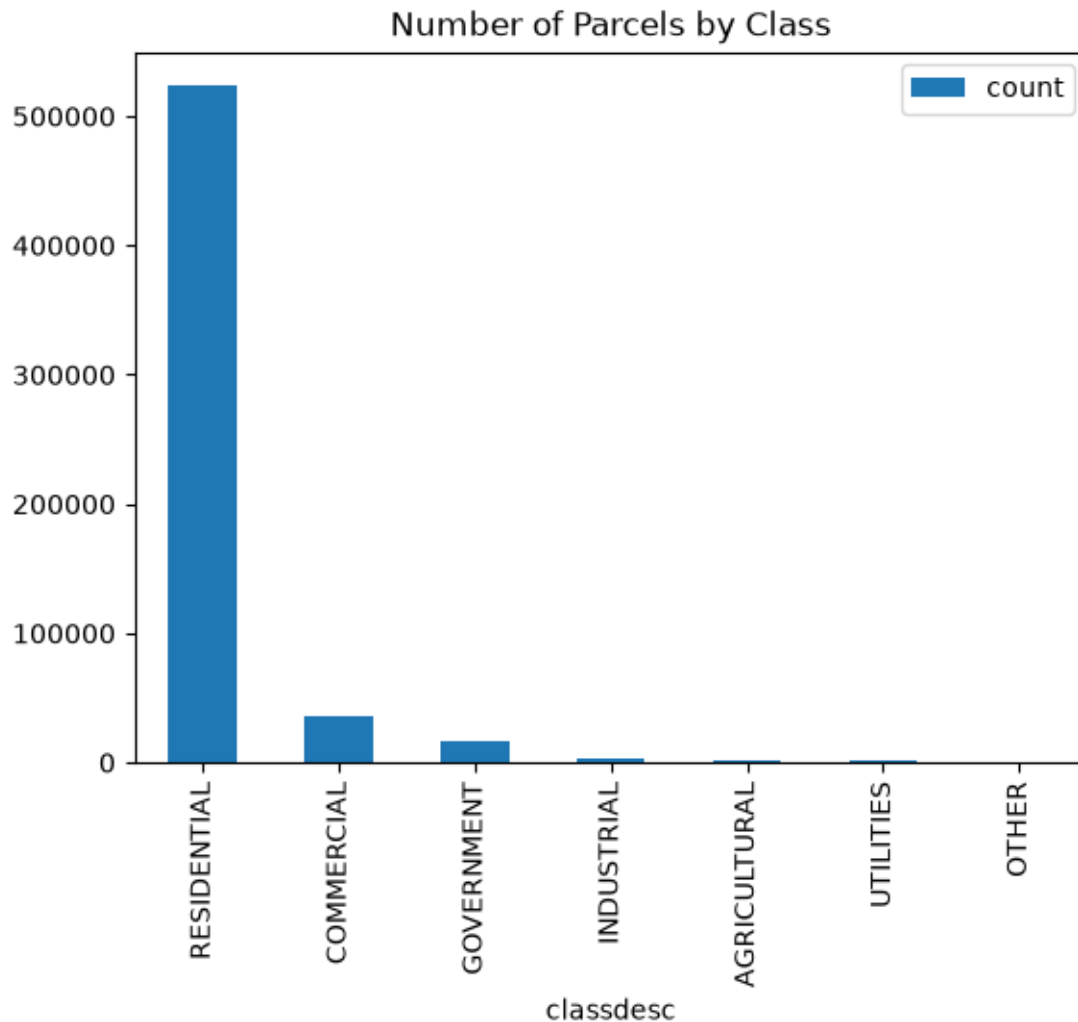
```

Top 3 parcel uses by class:

1. RESIDENTIAL (523754)
2. COMMERCIAL (35870)
3. GOVERNMENT (16881)

Top 3 parcel uses by class in 2024:

1. RESIDENTIAL (24161)
2. COMMERCIAL (1419)
3. INDUSTRIAL (142)



Most parcels are residential, which is perfect for our study because even when we filter to the dataset to just one year, we still get a significant number of residential parcels.

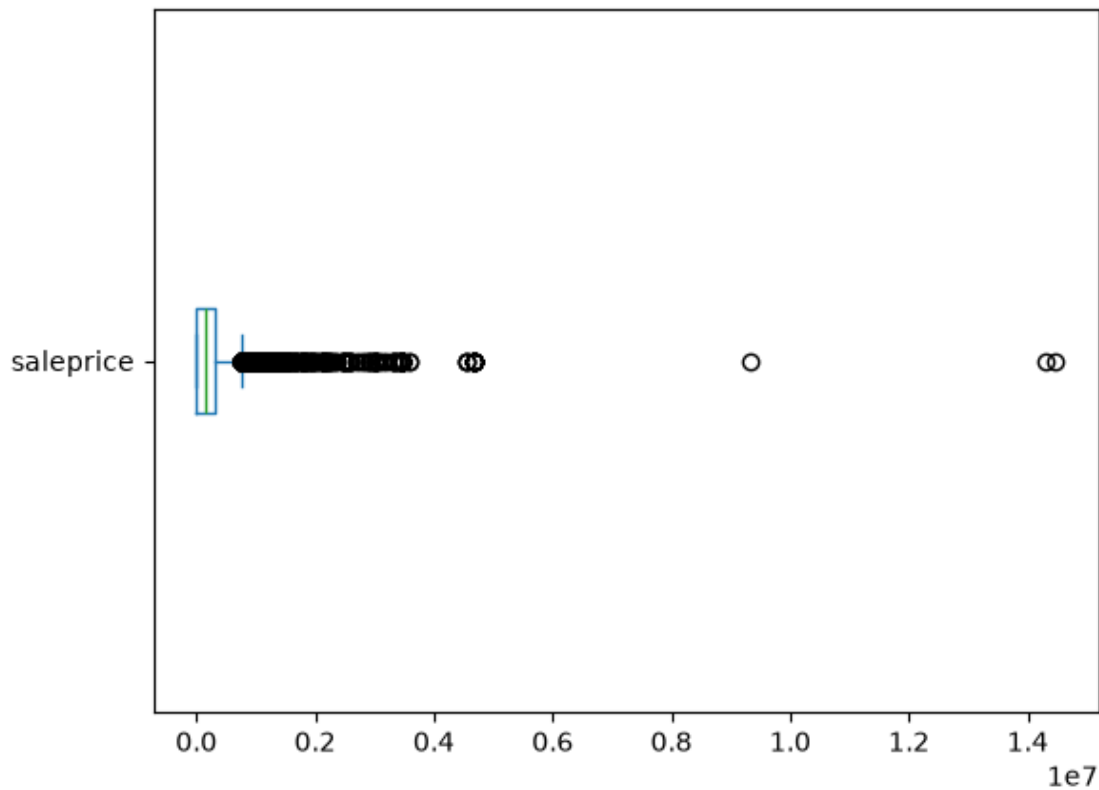
```
[5]: # Median residential sale value in 2024
summary_residential_sale_value_2024 = (
    assessment_parcel_2024_df[assessment_parcel_2024_df["classdesc"]=="RESIDENTIAL"]
    .agg({"saleprice":
        ["min", "max", "mean", "median"]})
    .round(2)
)
display(summary_residential_sale_value_2024)

assessment_parcel_2024_df[assessment_parcel_2024_df["classdesc"]=="RESIDENTIAL"]["saleprice"].
    plot.box(vert=False)
```

```
# create a boxplot of the residential sale values in 2024
# plt.figure(figsize=(10,6))
# plt.
    ↳boxplot(assessment_parcel_2024_df[assessment_parcel_2024_df["classdesc"]=="RESIDENTIAL"]["s
    ↳vert=False)
# plt.title("Boxplot of Residential Sale Values in 2024")
# plt.xlabel("Sale Price")
# plt.show()
```

```
saleprice
min      0.00
max     14466000.00
mean     220892.47
median   150000.00
```

[5]: <Axes: >



The median sale price for residential parcels is \$150,000 and the mean sale price is \$220,892, indicating that the data has a right skew. The maximum sale price \$14.466 million. This is a very high outlier, but there are a number of other outliers with properties sold over around \$5 and \$10 million. The minimum sale value is \$0. \$0 sales often mean homes were passed down or given away without sale, or foreclosed on. Major outliers and \$0 sales should be removed from the dataset.

```
[6]: assessment_parcel_2024_df_subset =
    ↪assessment_parcel_2024_df[(assessment_parcel_2024_df["classdesc"]=="RESIDENTIAL")
    ↪&

    ↪(assessment_parcel_2024_df["saleprice"]>0) &

    ↪(assessment_parcel_2024_df["saleprice"]<assessment_parcel_2024_df["saleprice"]
    ↪quantile(0.99))]

print(assessment_parcel_2024_df_subset.shape)

# summary of sale prices for residential properties in 2024 with 0s removed
summary_residential_sale_value_2024_subset = (assessment_parcel_2024_df_subset
    .filter(items=["saleprice"])
    .describe()
    .round(2)
    )

display(summary_residential_sale_value_2024_subset)
```

(23364, 53)

	saleprice
count	23364.00
mean	207498.67
std	243727.86
min	1.00
25%	10.00
50%	157500.00
75%	300000.00
max	2000000.00

When subsetting the data by removing \$0 sales at setting the max to less than the 99th percentile sale value, the median increases to \$157,500 and the mean decreases slightly to \$207,498. The number of rows drops by 797 to 23,364.

Next we will look at land, building, and total county and fair market values.

```
[7]: # Summary residential sale value in 2024
residential_property_value_2024_summary = (assessment_parcel_2024_df_subset
    .filter(items=["countybuilding",
    "countyland",
    "countytototal",
    "fairmarketbuilding",
    "fairmarketland",
    "fairmarkettototal"])
    .describe()
    .round(2))

residential_property_value_2024_summary
```

```
[7]:
```

	countybuilding	countyland	countyttotal	fairmarketbuilding \
count	23364.00	23364.00	23364.00	23364.00
mean	98864.94	30880.67	129745.60	106563.46
std	113665.97	31925.95	134680.65	115751.10
min	0.00	0.00	0.00	0.00
25%	32200.00	11500.00	48400.00	37900.00
50%	68200.00	23800.00	95800.00	77000.00
75%	127325.00	41600.00	166400.00	137500.00
max	4003800.00	1050000.00	4250000.00	4003800.00

	fairmarketland	fairmarketttotal
count	23364.00	23364.00
mean	31069.60	137633.05
std	32294.83	137349.64
min	0.00	0.00
25%	11700.00	53900.00
50%	23900.00	104900.00
75%	41900.00	177500.00
max	1050000.00	4250000.00

Next, we want to check how many residential properties are in Mon Valley Municipalities. There are 2,800 parcels within Mon Valley and 20,564 outside of Mon Valley.

```
[8]: assessment_parcel_2024_df_subset.groupby("mon_valley").size().
      ↪reset_index(name="count")
```

```
[8]:
```

	mon_valley	count
0	False	20564
1	True	2800

The average sale price and parcel value is higher outside of Mon Valley than within Mon Valley.

```
[9]: # summary stats of properties in Mon Valley vs not in Mon Valley
assessment_by_dummies_subset = (assessment_parcel_2024_df_subset
                                .filter(items=["mon_valley",
                                                "saleprice",
                                                "countybuilding",
                                                "countyland",
                                                "countyttotal",
                                                "fairmarketbuilding",
                                                "fairmarketland",
                                                "fairmarketttotal"])
                                .groupby("mon_valley")
                                .mean()
                                .round(2))

assessment_by_dummies_subset
```

```
[9]:
```

	saleprice	countybuilding	countyland	countyttotal	\
mon_valley					
False	218594.08	105600.69	32994.04	138594.73	
True	126010.85	49395.64	15359.46	64755.10	

	fairmarketbuilding	fairmarketland	fairmarketttotal
mon_valley			
False	113509.83	33181.53	146691.36
True	55547.32	15558.96	71106.28

1.4 1.2. Demographic data

In order to account for variations in property value that had nothing to do with pollution, we included a number of sociodemographic characteristics at the census tract level.

```
[10]: # maps and histograms describing distribution of:
# vacancy rates
# median household income
# share of white population
# share of population with college degree
# (we also map and plot median home value, which is used in the vacancy rate_
↳models)

# load the ACS data and the census tract boundaries
acs = pd.read_csv("full_acs_df.csv")
tracts = gpd.read_file("allegheny_tracts.geojson")

# make the geoid columns match so we can merge
tracts["geoid"] = tracts["GEOID"].astype("int64")
tracts = tracts[["geoid", "geometry"]]
acs = acs.drop(columns="geometry")
acs_tracts = tracts.merge(acs, on="geoid")

# make the share variables used in the regressions
acs_tracts["vacancy_rate"] = acs_tracts["vac_housing"] /_
↳acs_tracts["tot_housing"] * 100
acs_tracts["pop_white_share"] = acs_tracts["pop_white"] /_
↳acs_tracts["pop_total"] * 100
college = (acs_tracts["Bachelor's degree"] + acs_tracts["Master's degree"] +
acs_tracts["Professional school degree"] + acs_tracts["Doctorate_
↳degree"])
acs_tracts["college_degree_share"] = college / acs_tracts["Population age 25_
↳and over (education universe)"] * 100

# variables to plot and their labels
acs_vars = {
    "hhinc": "Median Household Income ($)",
```

```

    "home_val": "Median Home Value ($)",
    "vacancy_rate": "Vacancy Rate (%)",
    "pop_white_share": "Share White Population (%)",
    "college_degree_share": "Share with College Degree (%)",
}

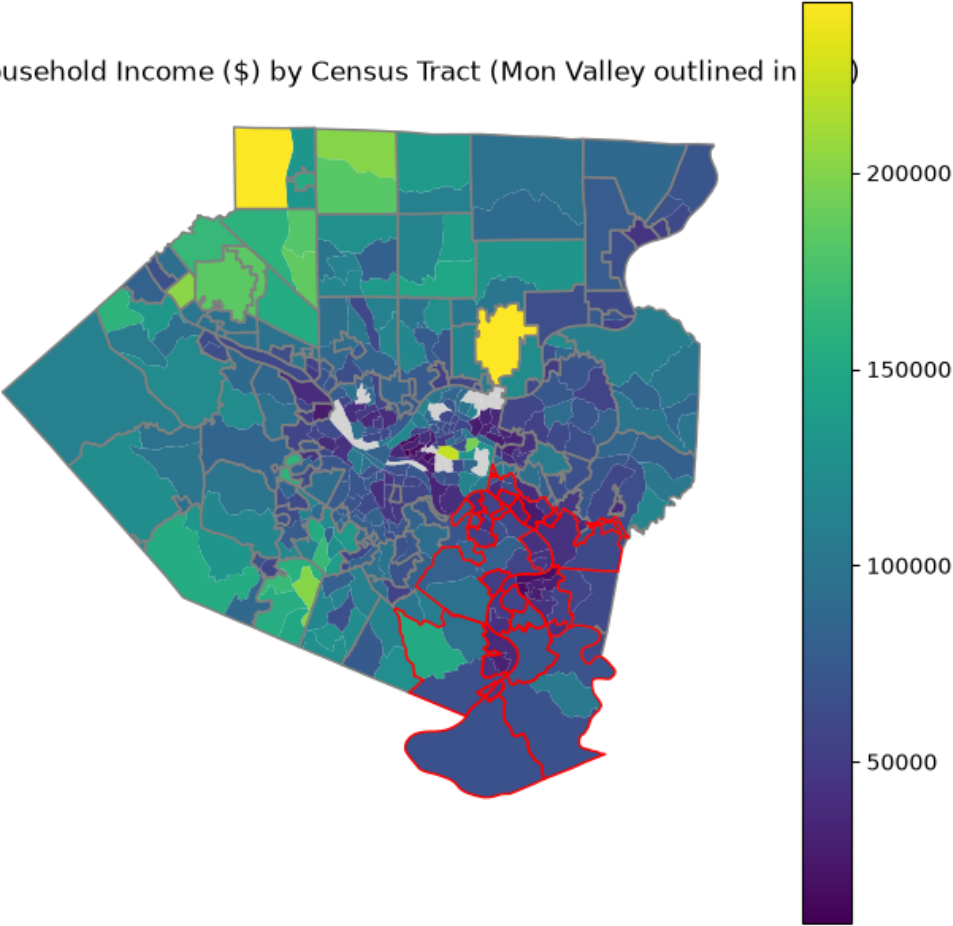
# municipality outlines, with mon valley municipalities flagged
# (same coordinate system as the tracts so they line up)
municipalities = gpd.read_file("allegHENY_municipal_boundaries.geojson")
municipalities = municipalities.to_crs(acs_tracts.crs)
municipalities["mon_valley"] = municipalities["NAME"].
    ↪isin(monvalley_municipalities)
mon_valley_munis = municipalities[municipalities["mon_valley"]]
other_munis = municipalities[~municipalities["mon_valley"]]

# static maps, one for each variable
# tracts are filled by the variable, municipality outlines go on top:
# mon valley in red, everything else in thicker gray
for col in acs_vars:
    fig, ax = plt.subplots(figsize=(8, 8))
    acs_tracts.plot(column=col, cmap="viridis", legend=True, ax=ax,
                    edgecolor="none", missing_kwds={"color": "lightgrey"})
    other_munis.plot(ax=ax, facecolor="none", edgecolor="gray", linewidth=1)
    mon_valley_munis.plot(ax=ax, facecolor="none", edgecolor="red", linewidth=1)
    ax.set_title(acs_vars[col] + " by Census Tract (Mon Valley outlined in_
    ↪red)")
    ax.set_axis_off()
    plt.show()

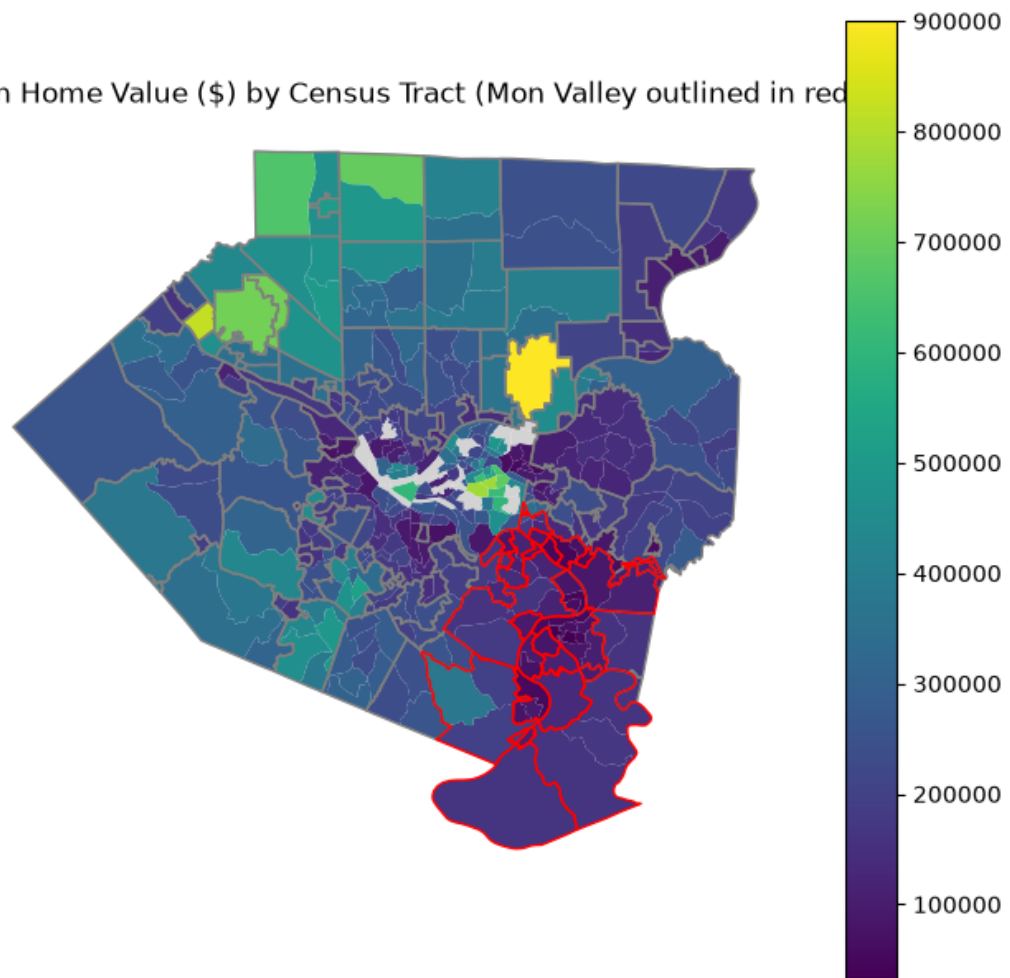
# histograms, one for each variable
for col in acs_vars:
    fig, ax = plt.subplots(figsize=(8, 5))
    ax.hist(acs_tracts[col].dropna(), bins=30)
    ax.set_title("Distribution of " + acs_vars[col])
    ax.set_xlabel(acs_vars[col])
    ax.set_ylabel("Number of Census Tracts")
    plt.show()

```

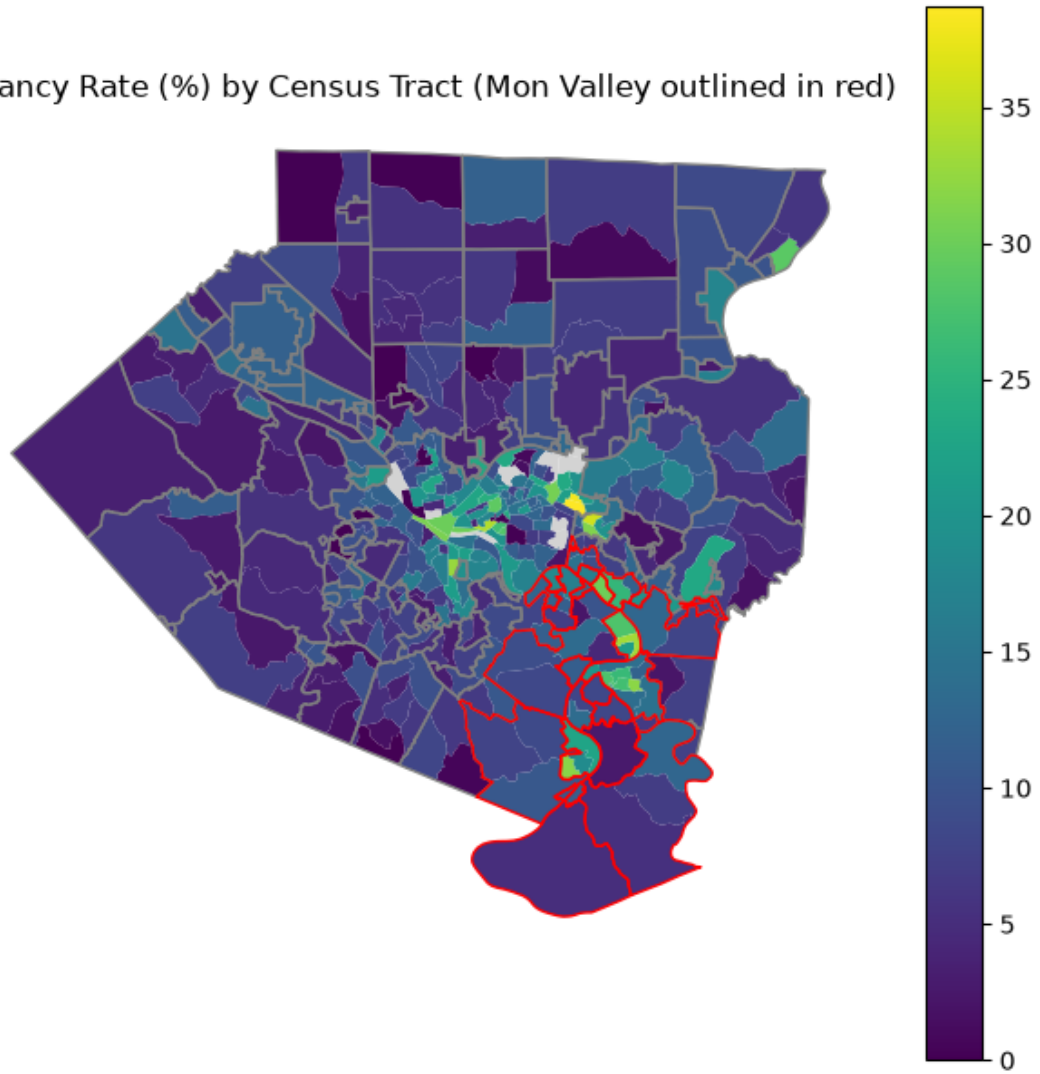
Median Household Income (\$) by Census Tract (Mon Valley outlined in red)



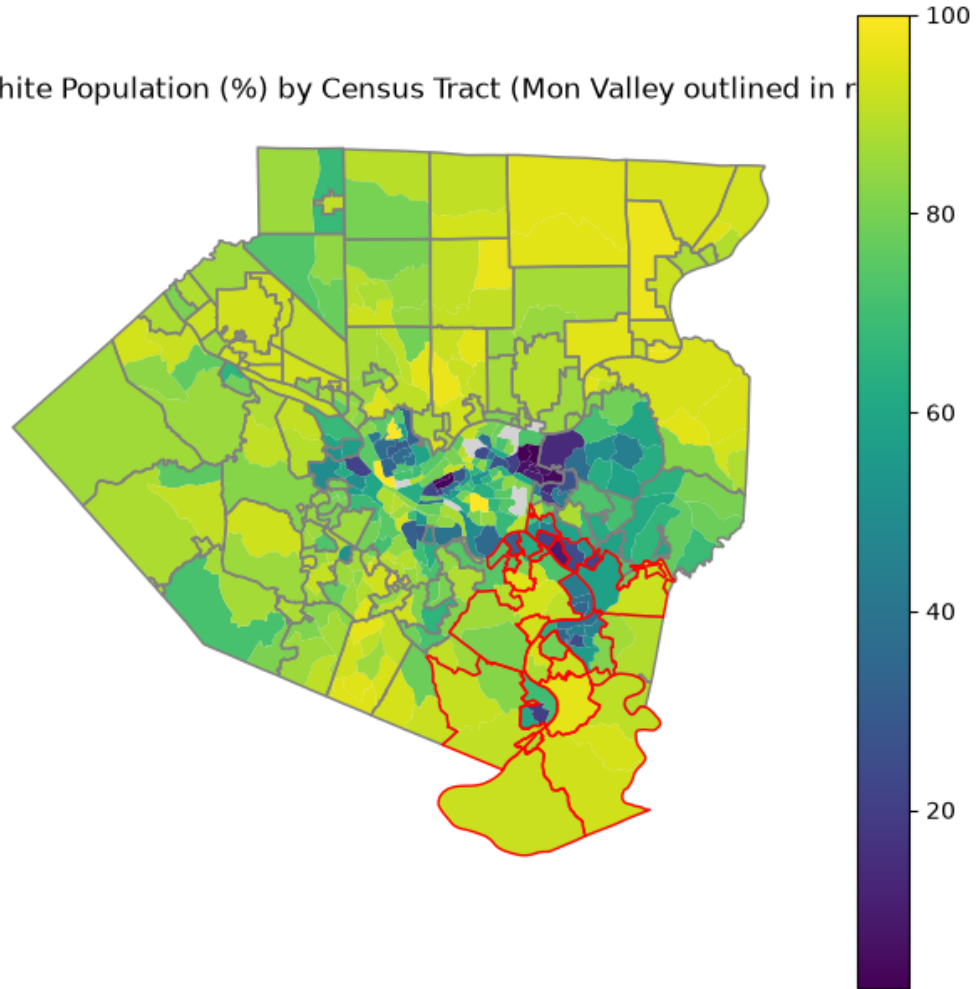
Median Home Value (\$) by Census Tract (Mon Valley outlined in red)



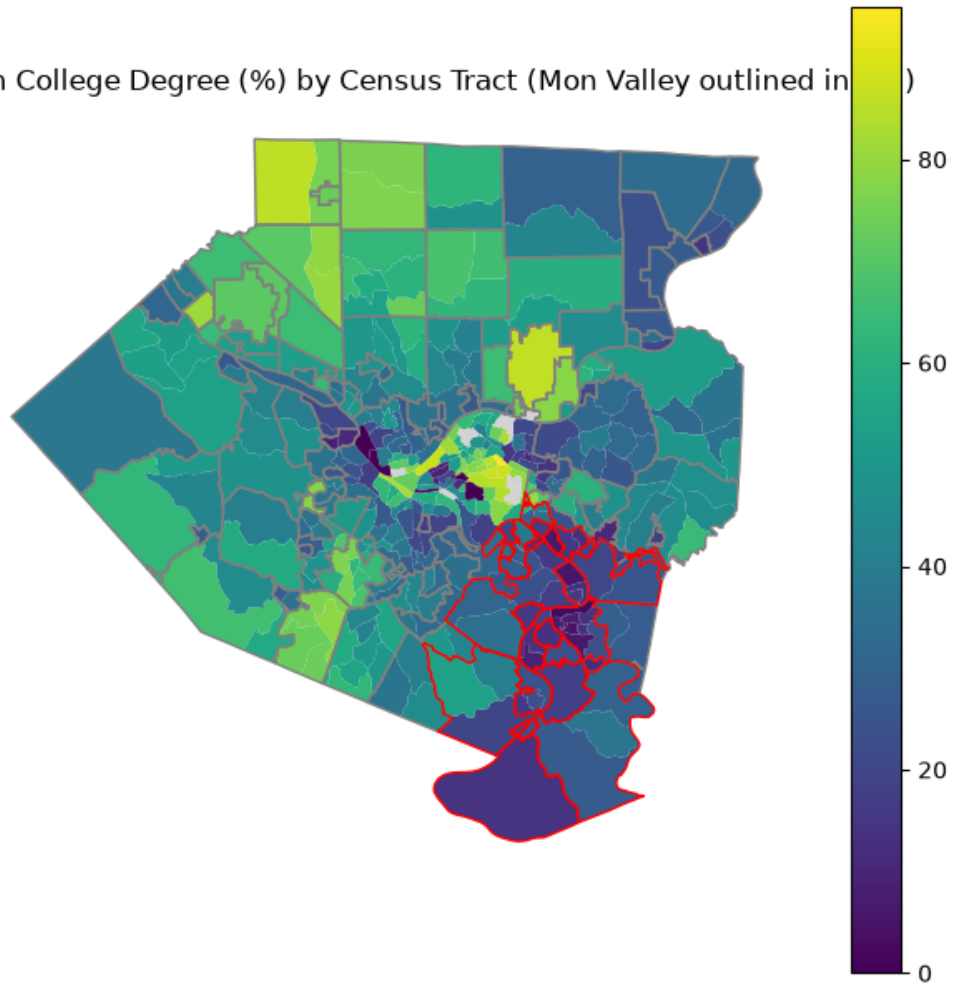
Vacancy Rate (%) by Census Tract (Mon Valley outlined in red)

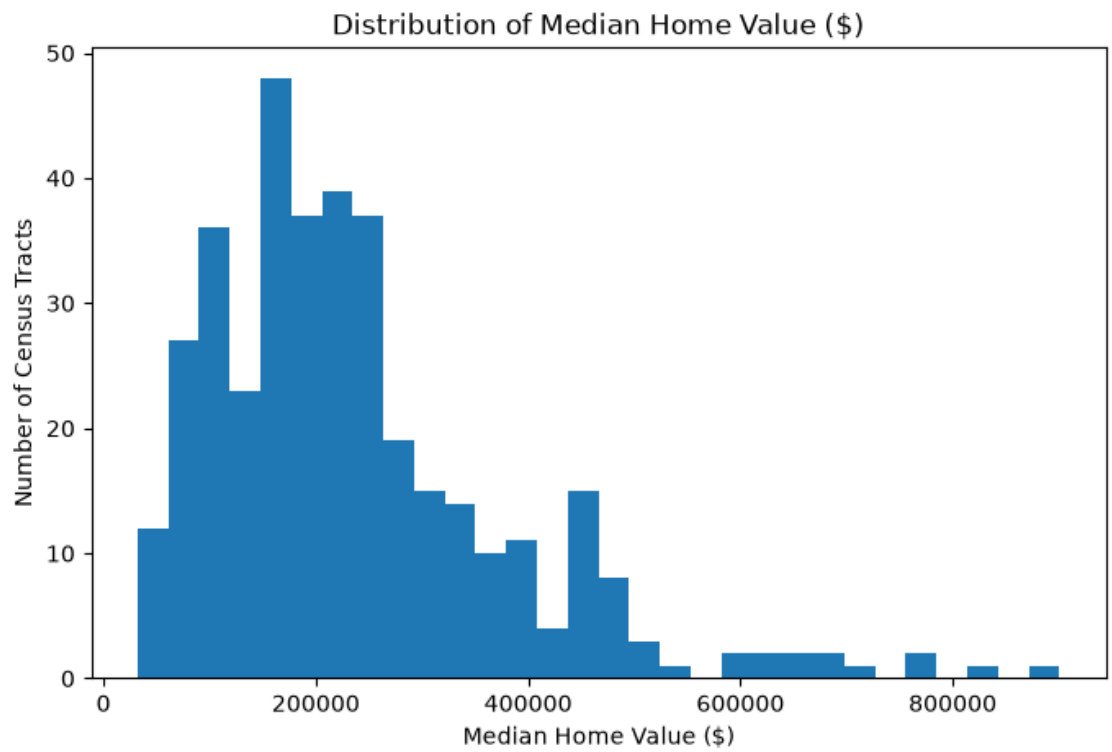
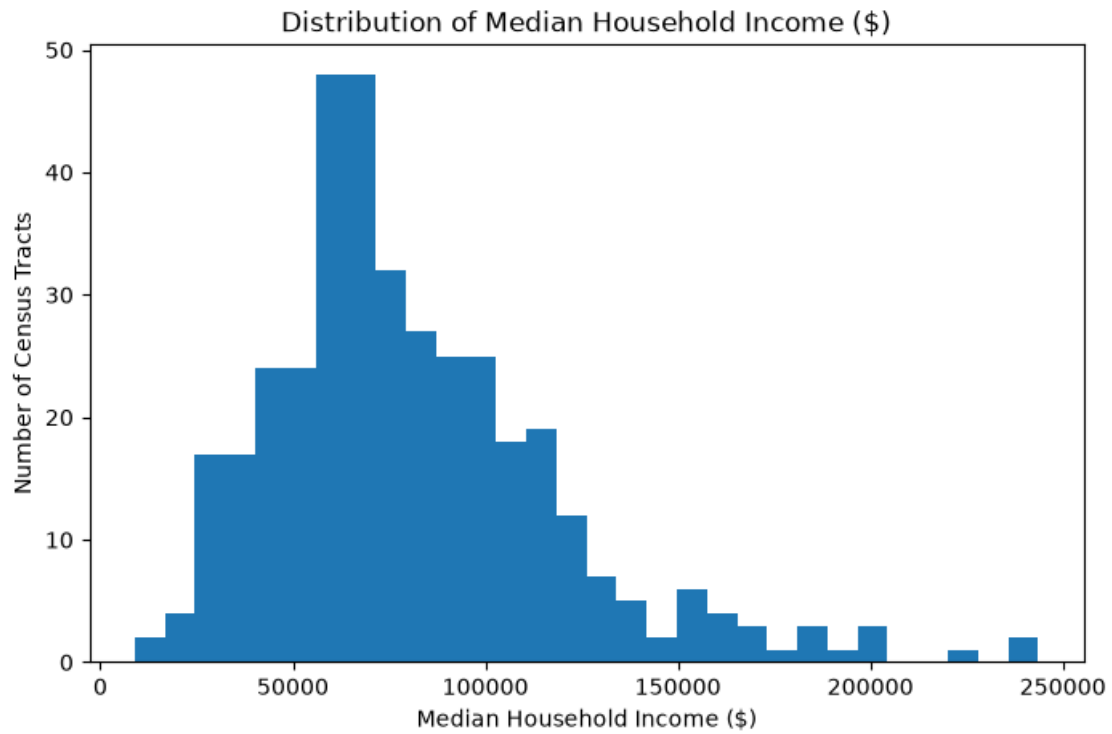


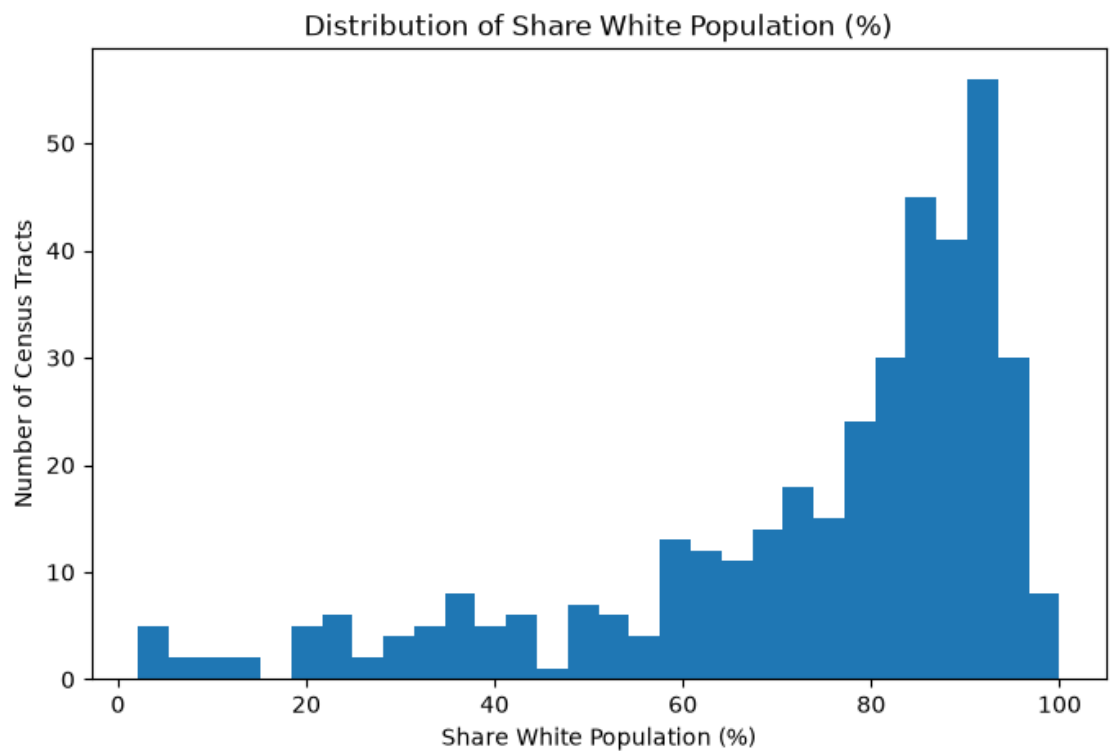
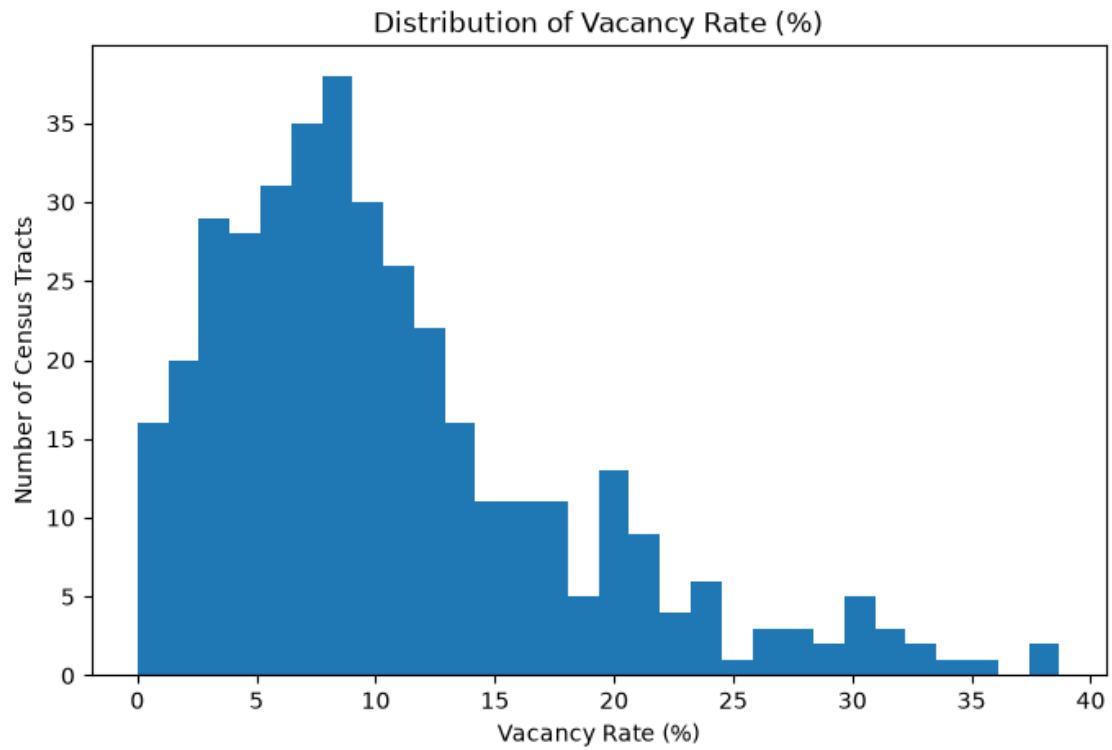
Share White Population (%) by Census Tract (Mon Valley outlined in red)

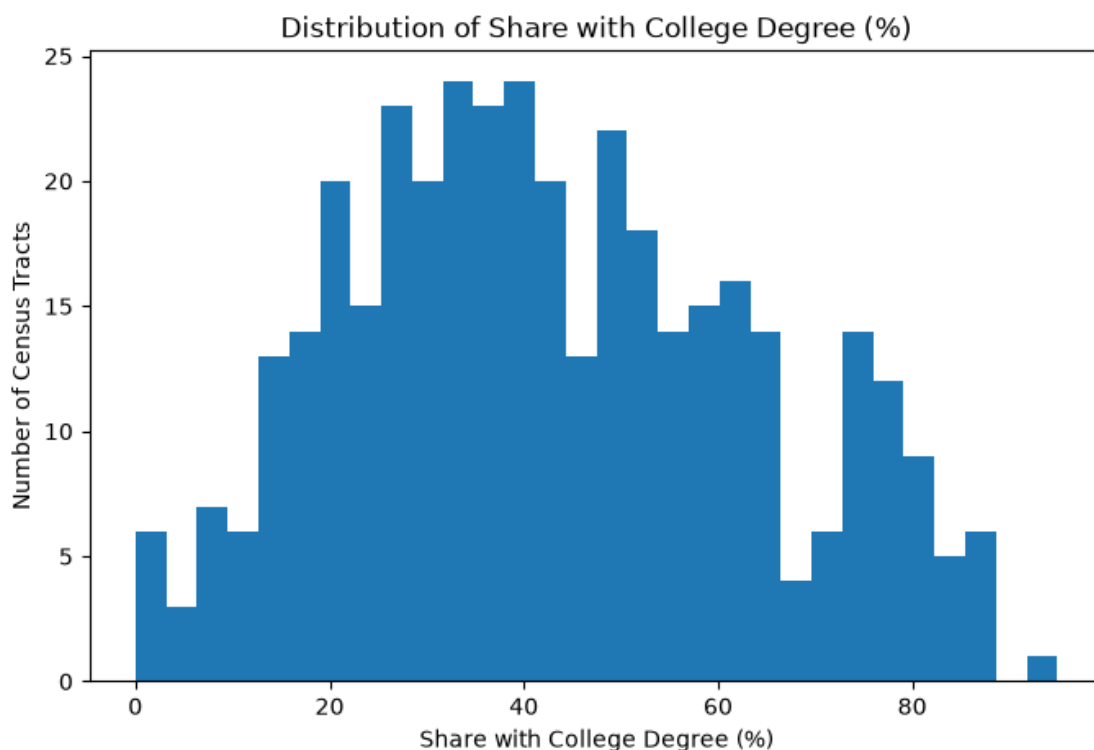


Share with College Degree (%) by Census Tract (Mon Valley outlined in red)









1.5 1.3. Pollution data

To evaluate the relationship between proximity to major industrial polluters and pollution exposure, we needed a list of “major industrial polluters.” Actually deciding which facilities qualify as “major” is tricky, because there are many ways to measure pollution. We initially considered restricting our analysis to the major Mon Valley Steel Mills based on the [CREATE Lab’s Mon Valley Report](#), further research into pollution in the Pittsburgh area found other significant sources that should be included, as in this 2018 analysis of the [Allegheny Toxic 10 list](#).

Exploring the datasets that these reports referenced, we eventually decided to draw on the the two official datasets most relevant to this question: the Toxics Release Inventory (TRI) (most recently released in 2024) and the Risk-Screening Environmental Indicators (RSEI) dataset (most recently released in 2022) produced by the EPA. These datasets measure related but different aspects of pollution: the TRI records the raw quantity of chemical releases, while RSEI weights releases by toxicity and population exposure potential as a unitless score by facility.

For each measure, we filtered the data to include only Allegheny County facilities and air releases (usually split into fugitive, meaning ambiently released, and stack, meaning controlled releases through smokestacks). We found that each dataset was heavily right skewed, with the vast majority of facilities reporting relatively low emissions or scores and a few facilities reporting extremely high values. Notably, the top emitters in each dataset were an overlapping but identical list of facilities. Based on this observation, we decided to define “major” polluters as those facilities that rank in the top class of either measure. Part of our consideration was that while the RSEI might better

represent absolute risk to human health, the TRI might better measure of emissions felt by nearby communities through smoke and odor.

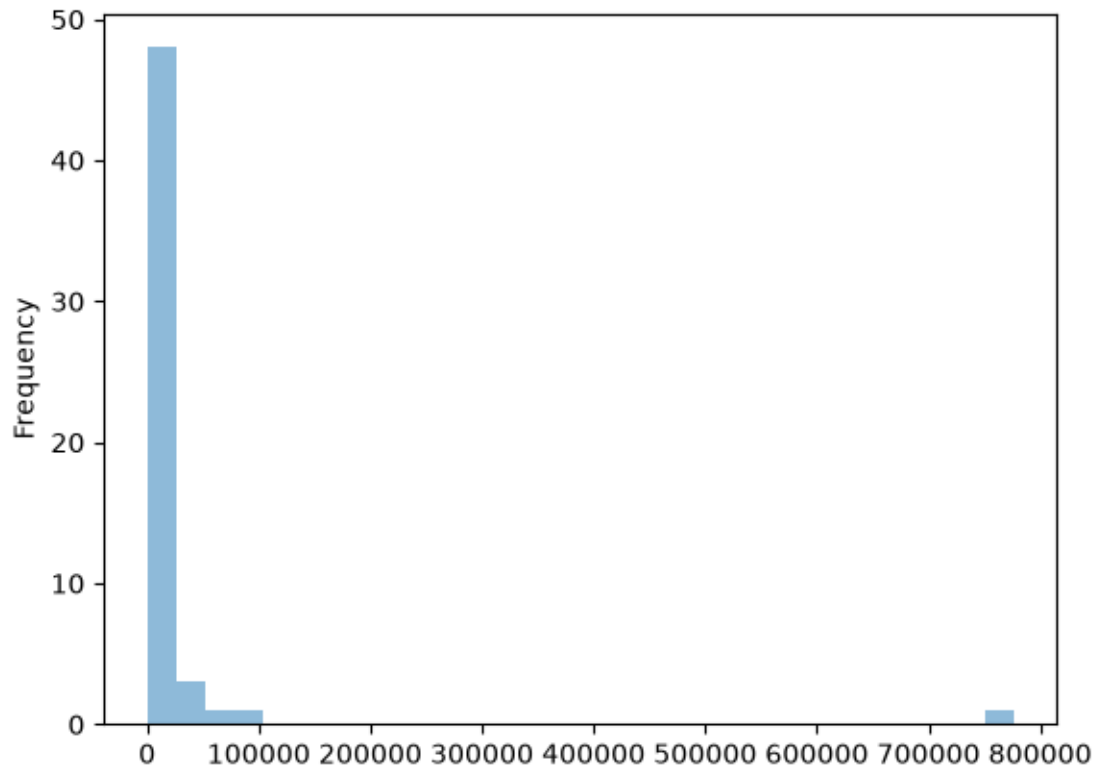
1.5.1 1.3.1. Toxics Release Inventory (TRI) Data

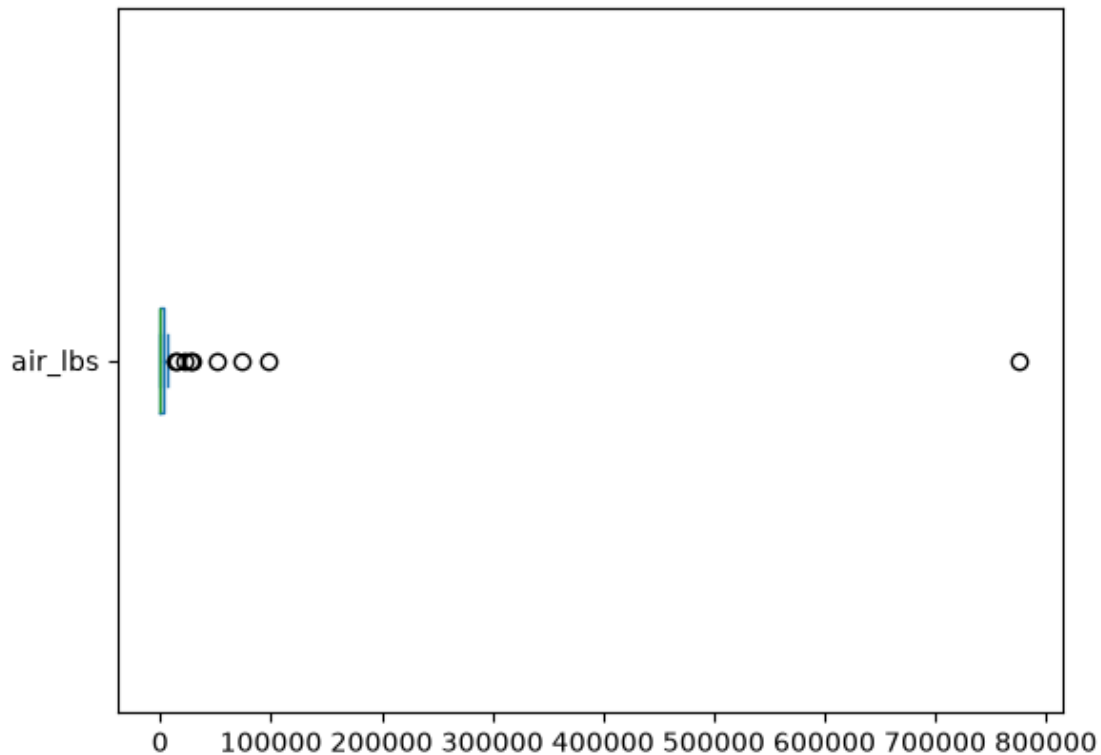
We used the [2024 TRI Basic Data File for Pennsylvania](#), which records the pounds of each listed chemical released by each facility. We filtered for Allegheny County facilities and air releases, summing fugitive and stack air releases to get a total air release per facility. Facilities reporting zero air releases were excluded. The resulting dataset included 54 facilities, 14 of which reported no air releases at all.

```
[11]: import pandas as pd
tri_basic=pd.read_csv("tri_2024_pa.csv")

tri_alleggh=(
    tri_basic
    .query("`7. COUNTY` == 'ALLEGHENY'")
    .filter(items=[
        "2. TRIFD", "4. FACILITY NAME", "5. STREET ADDRESS", "6. CITY",
        "12. LATITUDE", "13. LONGITUDE", "15. PARENT CO NAME",
        "50. UNIT OF MEASURE", "51. 5.1 - FUGITIVE AIR", "52. 5.2 - STACK AIR"
    ])
    .set_axis(["trifd", "name", "address", "city", "lat", "lon", "parent",
               "unit", "fugitive_air", "stack_air"], axis=1)
    .query("unit == 'Pounds'")
    .assign(air_lbs=lambda d: d["fugitive_air"].fillna(0) + d["stack_air"]
    .fillna(0))
    # because there are multiple entries per facility, we sum the air releases
    .after filling NAs with 0
    .groupby("trifd").agg(
        name=("name", "first"),
        address=("address", "first"),
        city=("city", "first"),
        parent=("parent", "first"),
        lat=("lat", "first"),
        lon=("lon", "first"),
        air_lbs=("air_lbs", "sum"),
    )
    .reset_index()
    .sort_values("air_lbs",
                  ascending=False)
    .reset_index(drop=True)
    .assign(tri_rank=lambda d: d.index + 1)
)
# tri_alleggh
```

```
[12]: # basic histogram and boxplot of TRI pounds
tri_alleggh[['air_lbs']].plot(kind='hist', bins=30, alpha=0.5, legend=False)
tri_alleggh[['air_lbs']].plot(kind='box', vert=False, legend=False)
plt.show()
```





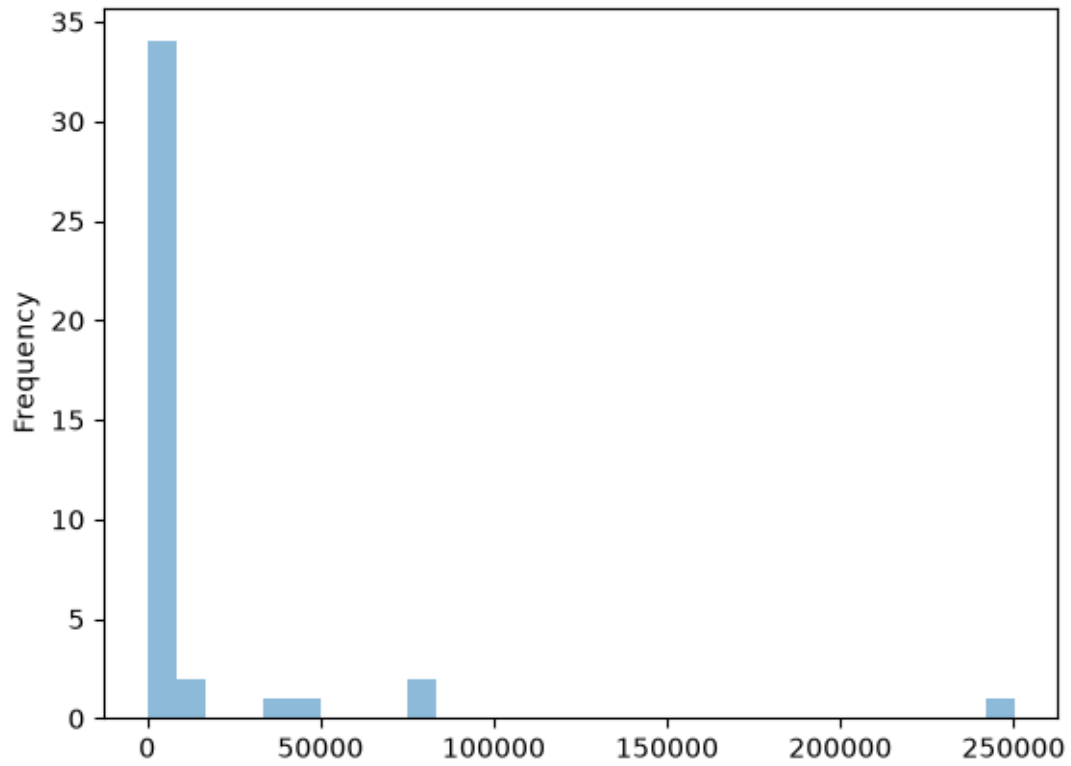
1.5.2 1.3.2 Risk Screening Environmental Indicators (RSEI) Scores

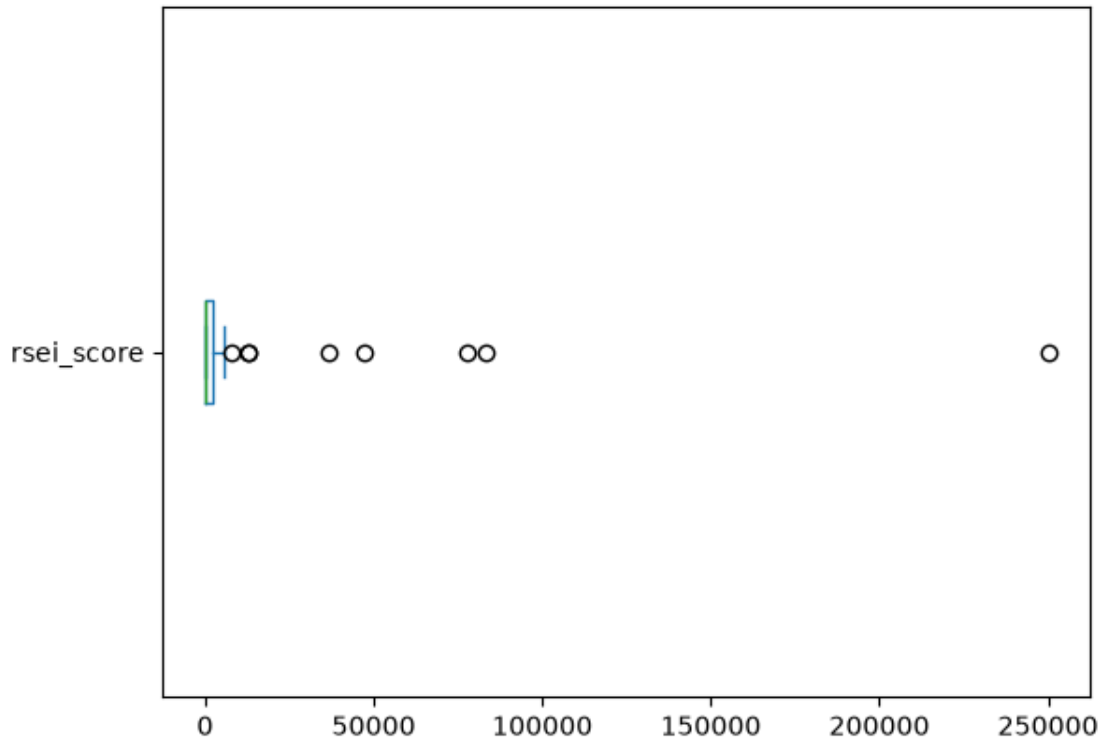
We downloaded RSEI scores for all facilities in Allegheny County from the [EasyRSEI dashboard](#) by navigating to the “Report” section on the left, selecting “Allegheny County” as the geographic area, navigating to the filter button at the top right and selecting only “Fugitive Air” and “Stack Air” releases, and exporting the resulting data as a CSV file.

```
[13]: easyrsei = (
    pd.read_excel("easyrsei_2022_air.xlsx")
    .rename(
        columns={
            "Facility Name": "name",
            "Street": "address",
            "RSEI Score": "rsei_score"
        }
    )
    .filter(items=[
        "name", "address", "rsei_score"
    ])
    .assign(rsei_rank=lambda d: d["rsei_score"].rank(ascending=False,
        ↪method="min"))
)
easyrsei
```

```
easyrsei[['rsei_score']].plot(kind='hist', bins=30, alpha=0.5, legend=False)  
easyrsei[['rsei_score']].plot(kind='box', vert=False, legend=False)
```

[13]: <Axes: >





1.5.3 1.3.3 Combining TRI and RSEI Scores

As is evident in the box plots and histograms of the RSEI scores, both datasets are extremely right skewed, with just a handful of facilities identified as outliers in the boxplot. Because the measures span several orders of magnitude, we sorted each one into classes on a log scale using Fisher–Jenks natural breaks from the `mapclassify` package. Fisher–Jenks splits values into a set number of groups so that values inside each group are as similar as possible, and we counted a facility as major if it landed in the top class of either measure. Using this process, we identified **11** major facilities; a full breakdown of the parameters we used to come to this number is available in section 2.3 of script `02_facility-locations.ipynb`. A table of facilities is shown below:

```
[14]: import mapclassify as mc

def top_class(values, k):
    """True for rows in the highest Jenks class (log10 scale). Zeros are never_
    ↪top."""
    out = pd.Series(False, index=values.index)
    pos = values[values > 0]
    jenks = mc.FisherJenks(np.log10(pos), k=k)
    out[pos.index] = jenks.yb == k - 1
    return out

tri_alleggh["tri_top"] = top_class(tri_alleggh["air_lbs"], k=5)
```

```

easyrsei["rsei_top"] = top_class(easyrsei["rsei_score"], k=4)

# Standardize addresses so the join isn't thrown off by spacing or
↳capitalization
for df in (tri_alleggh, easyrsei):
    df["address"] = df["address"].str.strip().str.upper()

major_fac = (
    tri_alleggh
    .merge(easyrsei.rename(columns={"name": "rsei_name"}),
           on="address", how="left", validate="one_to_one")
    .assign(rsei_top=lambda x: x["rsei_top"].fillna(False).astype(bool),
            major=lambda x: x["tri_top"] | x["rsei_top"])
    .query("major")
    .sort_values("air_lbs", ascending=False)
    .reset_index(drop=True)
)

# Stop if any RSEI top facility failed to match a TRI facility
unmatched = easyrsei.loc[easyrsei["rsei_top"] & ~easyrsei["address"].
↳isin(tri_alleggh["address"]), "name"]
assert unmatched.empty, f"RSEI top facilities with no TRI match: {unmatched.
↳tolist()}"

major_fac[["name", "address", "city", "tri_rank", "air_lbs", "rsei_rank",
↳"rsei_score"]].head(3)

```

C:\Users\Bryce\AppData\Local\Temp\ipykernel_22388\1458348887.py:11: UserWarning: Numba not installed. Using slow pure python version.

```
tri_alleggh["tri_top"] = top_class(tri_alleggh["air_lbs"], k=5)
```

C:\Users\Bryce\AppData\Local\Temp\ipykernel_22388\1458348887.py:12: UserWarning: Numba not installed. Using slow pure python version.

```
easyrsei["rsei_top"] = top_class(easyrsei["rsei_score"], k=4)
```

```
[14]:
```

	name	address \
0	USS-CLAIRTON PLANT	400 STATE ST MS 71
1	USS MON VALLEY WORKS - EDGAR THOMSON PLANT	BRADDOCK AVE & 13TH ST
2	USS MON VALLEY WORKS - IRVIN PLANT	CAMP HOLLOW RD

	city	tri_rank	air_lbs	rsei_rank	rsei_score
0	CLAIRTON	1	775872.72	2.0	83152.966207
1	BRADDOCK	2	96883.69	9.0	5536.456060
2	WEST MIFFLIN	3	73871.60	17.0	552.693876

```
[15]: # we still use a log of air_lbs because of how far Claireton Coke Works is from
↳the rest of the major facilities.
fig, axes = plt.subplots(1, 2, figsize=(12, 5))
```

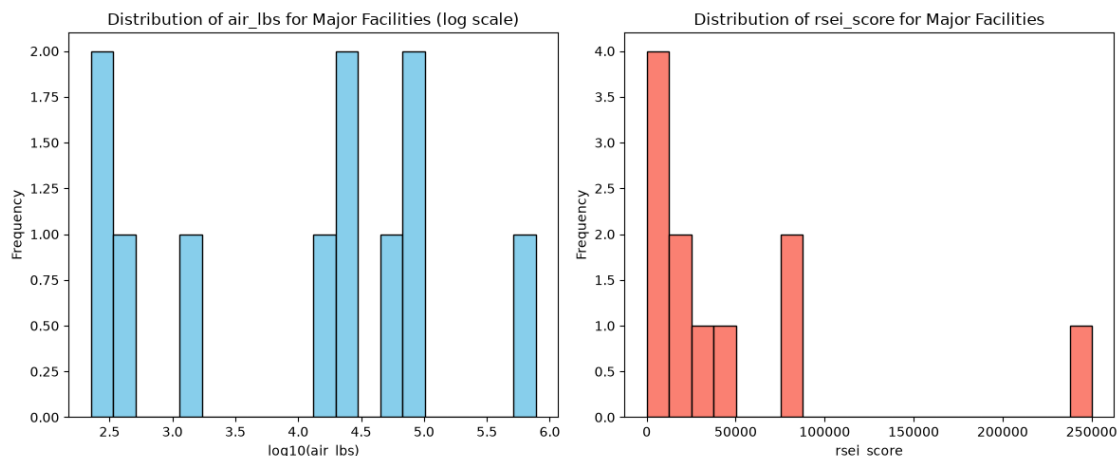
```

axes[0].hist(np.log10(major_fac["air_lbs"]), bins=20, color="skyblue",
             edgecolor="black")
axes[0].set_title("Distribution of air_lbs for Major Facilities (log scale)")
axes[0].set_xlabel("log10(air_lbs)")
axes[0].set_ylabel("Frequency")

axes[1].hist(major_fac["rsei_score"], bins=20, color="salmon",
             edgecolor="black")
axes[1].set_title("Distribution of rsei_score for Major Facilities")
axes[1].set_xlabel("rsei_score")
axes[1].set_ylabel("Frequency")

plt.tight_layout()
plt.show()

```



The only facilities in both top classes were Clairton and Edgar Thomson, both of which are associated with steelmaking in the Mon Valley. We matched the two files on standardized street address and checked that every RSEI top-class facility found a TRI match.

In a comparison between Environment America’s “[Allegheny County’s Toxic Ten](#)” (2023), our list captured nine of the ten facilities they identified. Missing from our list is Holtec Manufacturing in East Pittsburgh, which ranked 13th by 2022 RSEI score. Our list added the Irvin Works (also Mon Valley) and Synthomer Jefferson Hills chemical manufacturing plant, which is under a [\\$2.4 million consent decree](#) for widespread environmental regulations as of 2023.

The facility points come from the latitude and longitude reported in TRI (EPSG:4326). We projected them to EPSG:2272, NAD83 Pennsylvania South (US feet), and every distance below uses that projection.

1.5.4 1.3.4 Quantifying proximity of parcels to major polluters

We originally planned to use a simple “as the crow flies” distance from each parcel to the nearest facility. However, noting how concentrated facilities were along the Monongehela River south of

Pittsburgh, we decided we wanted to find a measure that accounted for proximity to multiple facilities. After a brief literature review, we landed on the creation of a distance-decay weighted index. This method adds up distance from all 11 facilities, but the measure of distance decays exponentially by a specified parameter. Stated formally, for parcel i :

$$E_i = \sum_{j=1}^{11} \frac{\log_{10}(\text{air}_j + 1)}{\log_{10}(\text{air} + 1)} e^{-d_{ij}/\lambda}$$

where d is the distance in miles from the parcel's boundary to facility j that released air pounds in 2024. In this model, a facility at a distance of 0 contributes about 37 percent of what it would at a distance of 0. At 3, it contributes about 5 percent, which is close to nothing.

We calculated exposure using two values: - 0.5 miles, which assumes very local effects, immediately noticeable when outdoors: odor, noise, visible stacks, soot on cars. - 1 mile, which assumes a somewhat wider reach. We based these values on [Currie, Davis, Greenstone, and Walker \(2015\)](#), which found that home values fell about 11% within half a mile of a newly opened toxic plant, with no detectable effect beyond a mile.

We decided to include the quantity of toxic air released as a weight in our model after reviewing the distribution of air releases and noting a persistent rightward skew. In our model, we weight each facility by $\log(\text{air lbs})$, which is scaled so the weights average 1, and a log is used to prevent Clairton from dominating the distribution. Clairton gets 1.47 and McConway & Torley gets 0.59, a ratio of 2.5, even though their reported pounds differ by a factor of about 3,400.

Code used to create this measure and a visualization of results is shown below:

```
[16]: # convert major facilities to geodataframe
facilities_gdf = gpd.GeoDataFrame(major_fac, geometry=gpd.
    ↪points_from_xy(major_fac["lon"], major_fac["lat"]),
    crs="EPSG:4326").to_crs("EPSG:2272")

# load parcels
parcels = gpd.read_file("assessor_parcels_analytic.geojson")

# load municipality boundaries
municipalities = gpd.read_file("allegheny_municipal_boundaries.geojson").
    ↪to_crs(epsg=2272)

dist = pd.DataFrame(
    # compute distance from each parcel to each facility in miles; originally
    ↪in feet
    {fid: parcels.geometry.distance(geom) / 5280 for fid, geom in
    ↪facilities_gdf.geometry.items()},
    index=parcels.index,
)

w_log = np.log10(facilities_gdf["air_lbs"] + 1)
w_log = w_log / w_log.mean() # mean weight of 1, same scale as unweighted
```

```

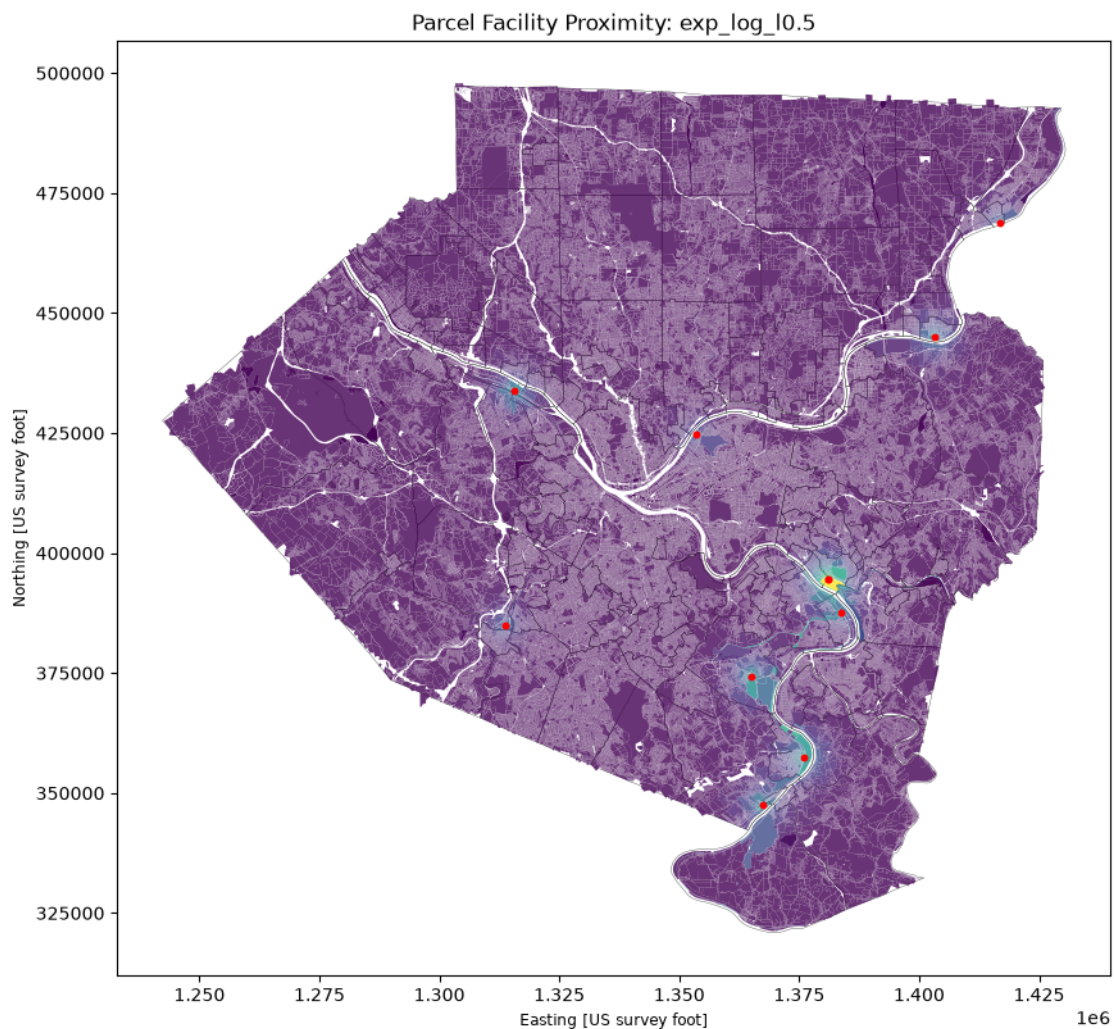
for lam in [0.5, 1]:
    decay = np.exp(-dist / lam)
    parcels[f"exp_log_{lam}"] = (decay * w_log).sum(axis=1)

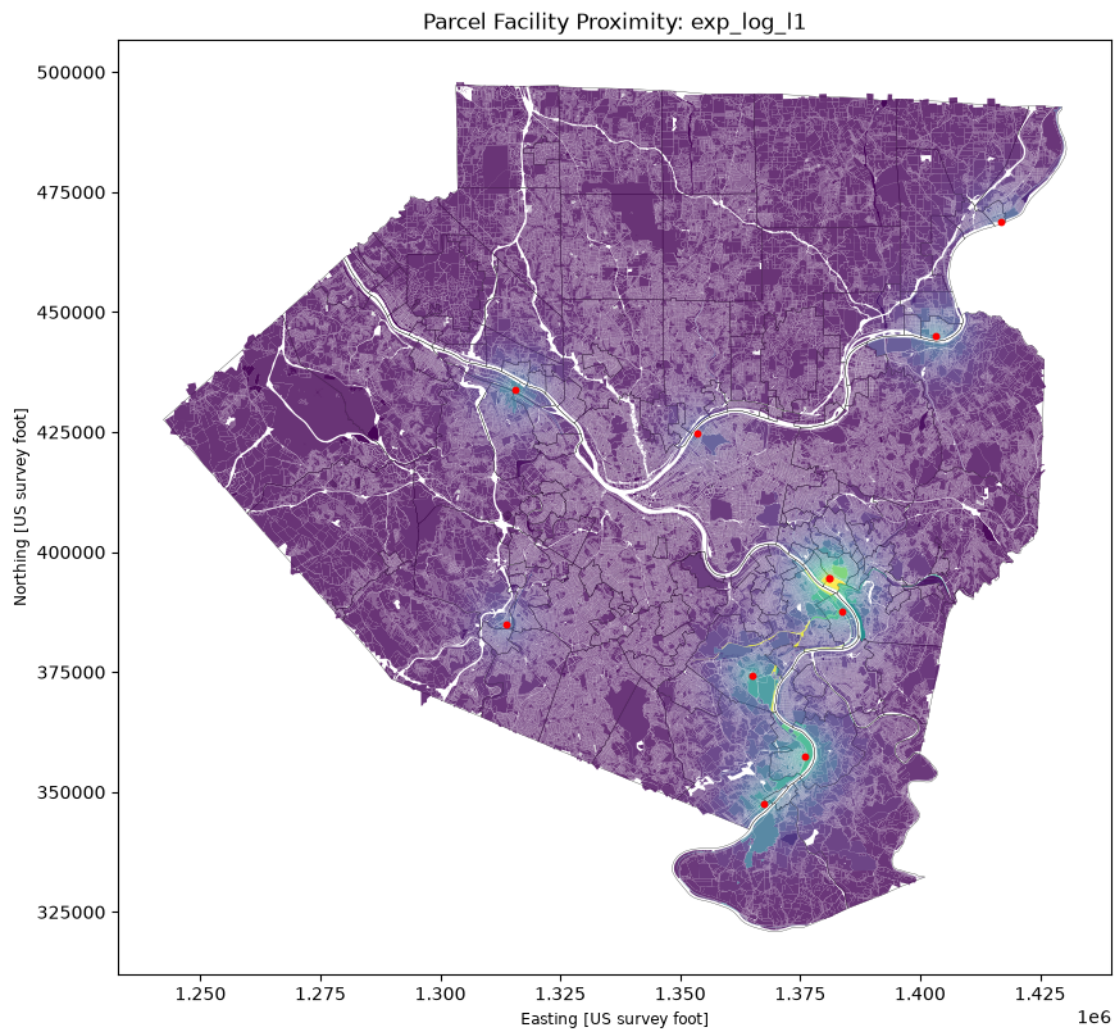
dist_cols = ['exp_log_10.5', 'exp_log_11']

for col in dist_cols:
    fig, ax = plt.subplots(figsize=(10, 10))
    municipalities.plot(ax=ax, color='none', edgecolor='black', linewidth=0.2)
    parcels.plot(ax=ax, column=col, cmap='viridis', markersize=5, alpha=0.8)
    facilities_gdf.plot(ax=ax, color='red', markersize=10)
    ax.set_title(f"Parcel Facility Proximity: {col}")
    plt.show()

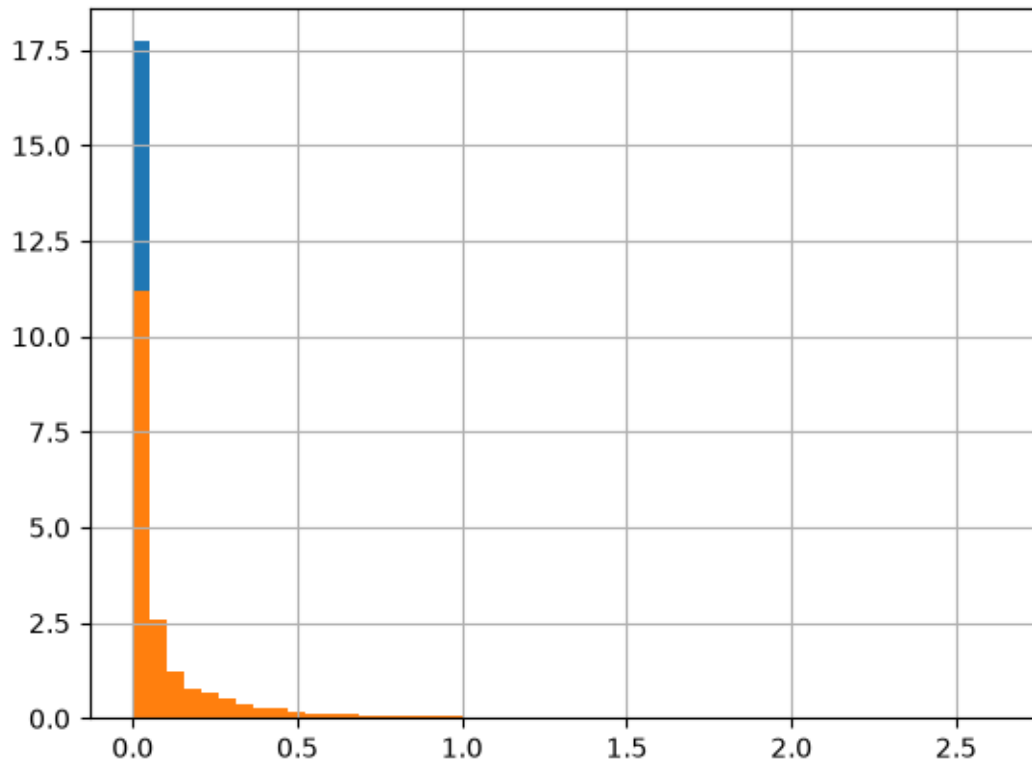
parcels["exp_log_10.5"].hist(bins=50, density=True)
parcels["exp_log_11"].hist(bins=50, density=True)

```





[16]: <Axes: >



While it isn't possible to capture every nuance of exposure at the individual parcel level, a shortcut is looking at the municipal level.

```
[17]: import folium

municipalities = municipalities.to_crs(parcels.crs)

# map each exposure column to the layer name shown in the map
layers = {
    "exp_log_10.5": "Weighted by log10(air lbs),  = 0.5 mi",
    "exp_log_11": "Weighted by log10(air lbs),  = 1 mi",
}
exposure_cols = list(layers)

# 1. Find which municipality each parcel is in
parcels_in_muni = gpd.sjoin(
    parcels[exposure_cols + ["geometry"]],
    municipalities[["NAME", "geometry"]],
    predicate="within",
)

# 2. Average exposure and parcel count for each municipality
```

```

muni_stats = parcels_in_muni.groupby("index_right")[exposure_cols].mean()
muni_stats["n_parcel"] = parcels_in_muni.groupby("index_right").size()
muni_map = municipalities.join(muni_stats)

# 3. Convert each average to a percentile (0-100) so the colors are comparable
for col in exposure_cols:
    muni_map[col + "_pct"] = muni_map[col].rank(pct=True) * 100

# 4. Build the map: one toggleable layer per exposure column
m = folium.Map()

for col, layer_name in layers.items():
    muni_map.explore(
        m=m,
        column=col + "_pct",
        cmap="magma_r",
        vmin=0,
        vmax=100,
        name=layer_name,
        show=(col == "exp_log_10.5"), # only this layer is on by default
        legend=(col == "exp_log_10.5"),
        legend_kws={"caption": "Municipal mean exposure, percentile"},
        tooltip=["NAME", col, "n_parcel"],
        style_kws={"weight": 0.5, "color": "grey", "fillOpacity": 0.7},
    )

# 5. Add the facilities and save
facilities_gdf.explore(
    m=m,
    name="Major facilities",
    color="cyan",
    marker_kws={"radius": 7},
    style_kws={"color": "black", "weight": 1},
    tooltip=["name", "air_lbs"],
)
folium.LayerControl(collapsed=False).add_to(m)
m.fit_bounds(m.get_bounds())
m.save("exposure_municipal.html")
m

```

[17]: <folium.folium.Map at 0x1fac02bae40>

1.6 2. Regression Models

We will model two primary regressions: - Residential parcel values (sale prices) by the proximity to major polluters - Covariates will include median household income, vacancy rates, share white, college educated share, and a dummy for Mon Valley - Null hypothesis: proximity to major industrial polluters has no effect on home values - Census tract level vacancy rates by the proximity to

major polluters - Covariates will include median home value, share white, college educated share, and a dummy for Mon Valley - Null hypothesis: proximity to major industrial polluters has no effect on vacancy rates

Our first model is as follows:

$$\text{ResidentialSalePrice}_i = \beta_0 + \beta_1 \text{ProximityToPolluter}_i + \beta_2 \text{MedianHouseholdIncome}_i + \beta_3 \text{VacancyRate}_i + \beta_4 \text{ShareWhite}_i$$

And model two is:

$$\text{Vacancy Rate}_i = \beta_0 + \beta_1 \text{ProximityToPolluter}_i + \beta_2 \text{MedianHomeValue}_i + \beta_3 \text{ShareWhite}_i + \beta_4 \text{ShareCollegeEducated}_i$$

Because the models come in pairs, the residential parcel sale values and vacancy rate models will be modified to add an interaction term between ProximityToPolluter and the MonValley dummy variable. For example, model 1 with the interaction term would be:

$$\text{ResidentialSalePrice}_i = \beta_0 + \beta_1 \text{ProximityToPolluter}_i + \beta_2 \text{MedianHouseholdIncome}_i + \beta_3 \text{VacancyRate}_i + \beta_4 \text{ShareWhite}_i$$

For each model, we will first estimate a baseline OLS regressions, then estimate the full model pairs with all covariates.

```
[18]: # Subset analysis_gdf to remove outlier sale prices
analysis_gdf_no_outliers = analysis_gdf[
    (analysis_gdf["saleprice"] < analysis_gdf["saleprice"].quantile(0.99)) &
    (analysis_gdf["saleprice"] > 0)
]
```

1.6.1 Model 1: OLS Regression on Sale Prices with and without Mon Valley Interaction

```
[19]: # view trends in data
# plt.scatter(model_data["vacancy_rate"], model_data["saleprice"])
# x = model_data["pop_white_share"]
# y = model_data["saleprice"]

# coeffs = np.polyfit(x, y, 1)
# plt.plot(x, np.polyval(coeffs, x), color="red")
# plt.xlabel("Vacancy Rate")
# plt.ylabel("Sale Price")
# plt.title("Sale Price vs Vacancy Rate")
# plt.show()
```

1.6.2 Baseline OLS Regression on Sale Prices

```
[20]: feature_cols = ["exp_log_l1", "vacancy_rate", "hhinc", "pop_white_share",
    ↪ "college_degree_share"]
model_data = analysis_gdf_no_outliers.loc[analysis_gdf_no_outliers["pop_total"]
    ↪ > 0, ["saleprice", "tractfips", "mon_valley", *feature_cols]].apply(pd.
    ↪ to_numeric, errors="coerce").dropna()

fe_model_baseline = pf.feols("saleprice ~ exp_log_l1 + mon_valley + exp_log_l1:
    ↪ mon_valley", data=model_data, vcov="HC1")

fe_model_baseline.coef()
fe_model_baseline.se()
fe_model_baseline.tidy()

# pf.etable(
#     fe_model_baseline,
#     labels={
#         "exp_log_l1": "(Weighted) Proximity Index (exp_log_l1)",
#         "mon_valley": "Mon Valley Dummy"
#     },
#     caption="Residential Property Sale Price: Baseline OLS"
# )
```

```
[20]:
```

	Estimate	Std. Error	t value	Pr(> t)	\
Coefficient					
Intercept	227345.699568	1895.021377	119.969992	0.0	
exp_log_l1	-191909.527753	10214.350580	-18.788226	0.0	
mon_valley	-85807.302826	4772.580078	-17.979227	0.0	
exp_log_l1:mon_valley	104019.749921	12116.705202	8.584821	0.0	
	2.5%	97.5%			
Coefficient					
Intercept	223631.332353	231060.066782			
exp_log_l1	-211930.330349	-171888.725158			
mon_valley	-95161.875381	-76452.730270			
exp_log_l1:mon_valley	80270.206467	127769.293376			

Before running our primary model, we estimate baseline results. This baseline regression includes the weighted proximity to polluters index variable, the dummy variable for whether or not a property is located in Mon Valley, and an interaction term. This result establishes that closer proximity to a polluter decreases home values and being located in Mon Valley decreases home values. But, when interacting the two covariates, the effect of closer proximity to polluters lessens, though is still negative.

Below, we estimate our full model for Model 1.

1.6.3 Full Model 1

```
[21]: fe_model = pf.feols("saleprice ~ exp_log_l1 + mon_valley + vacancy_rate + hhinc +
  ↪pop_white_share + college_degree_share", data=model_data, vcov="HC1")

fe_model.coef()
fe_model.se()
fe_model.tidy()
```

```
[21]:
```

	Estimate	Std. Error	t value	Pr(> t)	\
Coefficient					
Intercept	-81584.625625	7783.721377	-10.481442	0.000000e+00	
exp_log_l1	-34253.432880	5467.560807	-6.264847	3.797416e-10	
mon_valley	958.017241	3565.369502	0.268701	7.881624e-01	
vacancy_rate	90134.333053	23872.983494	3.775579	1.600339e-04	
hhinc	1.649661	0.075909	21.731961	0.000000e+00	
pop_white_share	-14879.373553	7618.887816	-1.952959	5.083648e-02	
college_degree_share	358173.130184	18886.758644	18.964246	0.000000e+00	
	2.5%	97.5%			
Coefficient					
Intercept	-96841.234351	-66328.016899			
exp_log_l1	-44970.213696	-23536.652065			
mon_valley	-6030.342801	7946.377283			
vacancy_rate	43341.706409	136926.959698			
hhinc	1.500874	1.798449			
pop_white_share	-29812.897598	54.150493			
college_degree_share	321153.834041	395192.426326			

Model 1 excluding the interaction: This model indicates that closer proximity to a industrial polluter decreased the sale price of residential properties in 2024. For every 1 unit increase in the proximity index, sale prices decrease by \$34,253. This result is statistically significant at the 1 percent level. Interestingly, being located in the Mon Valley increases home sale prices by about \$958. Also surprisingly, a 1 percent increase in vacancy rates increases the sale price by over \$90,000 in this model. However, the above scatter plot indicates that the maximum vacancy rate is not above 40 percent. A linear model may not accurately capture the effect of this variable.

```
[22]: # model with interaction term between exp_log_l1 and mon_valley
fe_model2 = pf.feols("saleprice ~ exp_log_l1 + mon_valley + vacancy_rate +
  ↪hhinc + pop_white_share + college_degree_share + exp_log_l1:mon_valley",
  ↪data=model_data, vcov="HC1")

fe_model2.coef()
fe_model2.se()
fe_model2.tidy()
```

[22]:	Estimate	Std. Error	t value	Pr(> t)	\
Coefficient					
Intercept	-82161.655945	7772.788438	-10.570422	0.000000e+00	
exp_log_l1	-49514.512508	9652.356291	-5.129785	2.924053e-07	
mon_valley	-5946.603701	4453.396698	-1.335296	1.817925e-01	
vacancy_rate	93917.643217	23894.696469	3.930481	8.502187e-05	
hhinc	1.640601	0.076229	21.521919	0.000000e+00	
pop_white_share	-12169.210077	7816.382994	-1.556885	1.195114e-01	
college_degree_share	358321.198184	18881.371335	18.977499	0.000000e+00	
exp_log_l1:mon_valley	29445.587516	11308.417931	2.603864	9.223787e-03	
	2.5%	97.5%			
Coefficient					
Intercept	-97396.835421	-66926.476468			
exp_log_l1	-68433.769303	-30595.255713			
mon_valley	-14675.555803	2782.348402			
vacancy_rate	47082.457599	140752.828834			
hhinc	1.491187	1.790016			
pop_white_share	-27489.837768	3151.417614			
college_degree_share	321312.461441	395329.934927			
exp_log_l1:mon_valley	7280.340366	51610.834667			

Model 1 with interaction: This model interacts the weighted distance to industrial polluters variable with the Mon Valley dummy. Here, for every 1 percent increase in the proximity index, sale prices decrease by \$49,514 when not located in Mon Valley. However, when a residential property is located in Mon Valley, effect of closer proximity to a polluter only decreases sale price by \$20,069. Both results are still statistically significant. Additionally, being located in Mon Valley decreases sale prices by \$5,946. While we expected to see that being located closer to polluters decreased sale values for residential properties, we did not expect the effect to be weaker for properties located in Mon Valley.

Notably, neither model is particularly strong. The R^2 value for both columns is 0.191.

```
[23]: # pf.etable(
#       [fe_model, fe_model2],
#       labels={
#           "exp_log_l1": "(Weighted) Proximity Index (exp_log_l1)",
#           "mon_valley": "Mon Valley Dummy",
#           "vacancy_rate": "Vacancy Rate",
#           "hhinc": "Median Household Income",
#           "pop_white_share": "Share of White Population",
#           "college_degree_share": "Share of Population with College Degree",
#           "exp_log_l1:mon_valley": "exp_log_l1 x Mon Valley Dummy"
#       },
#       caption="Residential Property Sale Price: Naive OLS vs Interaction"
# )
```

1.6.4 Model 2: OLS Regression on Vacancy Rates with and Without Interaction

1.6.5 Baseline OLS Regression

```
[24]: feature_cols_2 = ["exp_log_l1", "vacancy_rate", "hhinc", "home_val",
    ↪ "pop_white_share", "college_degree_share"]
model_data2 = analysis_gdf_no_outliers.
    ↪ loc[analysis_gdf_no_outliers["pop_total"] > 0, ["saleprice", "mon_valley",
    ↪ *feature_cols_2]].apply(pd.to_numeric, errors="coerce").dropna()

fe_model2_baseline = pf.feols("vacancy_rate ~ exp_log_l1 + mon_valley +
    ↪ exp_log_l1:mon_valley", data=model_data2, vcov="HC1")

fe_model2_baseline.coef()
fe_model2_baseline.se()
fe_model2_baseline.tidy()

# pf.etable(
#     fe_model2_baseline,
#     labels={
#         "exp_log_l1": "(Weighted) Proximity Index (exp_log_l1)",
#         "mon_valley": "Mon Valley Dummy",
#         "exp_log_l1:mon_valley": "exp_log_l1 x Mon Valley Dummy"
#     },
#     caption="Vacancy Rate: Baseline OLS"
# )
```

```
[24]:
```

	Estimate	Std. Error	t value	Pr(> t)	\
Coefficient					
Intercept	0.084629	0.000550	153.799734	0.000000e+00	
exp_log_l1	0.096621	0.004706	20.530377	0.000000e+00	
mon_valley	0.014708	0.001901	7.739011	1.043610e-14	
exp_log_l1:mon_valley	-0.033602	0.006180	-5.437232	5.466220e-08	
	2.5%	97.5%			
Coefficient					
Intercept	0.083551	0.085708			
exp_log_l1	0.087397	0.105846			
mon_valley	0.010983	0.018433			
exp_log_l1:mon_valley	-0.045716	-0.021489			

The baseline regression of model 2 shows that increase proximity to polluters and being located in Mon Valley increases vacancy rates in neighborhoods. However, like in the baseline result for model 1, interacting distance to polluters with being located in Mon Valley lowers the rate that vacancy rates increase.

Below we estimate our full model for Model 2.

1.6.6 Full Model 2

```
[25]: # OLS Regression
fe_model3 = pf.feols("vacancy_rate ~ exp_log_l1 + mon_valley + home_val +
  ↪pop_white_share + college_degree_share", data=model_data2, vcov="HC1")
fe_model3.coef()
fe_model3.se()
fe_model3.tidy()
```

```
[25]:
```

	Estimate	Std. Error	t value	Pr(> t)	\
Coefficient					
Intercept	2.595473e-01	2.373201e-03	109.365927	0.000000e+00	
exp_log_l1	3.976294e-02	2.572014e-03	15.459846	0.000000e+00	
mon_valley	7.009210e-04	1.269793e-03	0.551996	5.809563e-01	
home_val	-2.901297e-08	4.565823e-09	-6.354379	2.131695e-10	
pop_white_share	-1.915597e-01	2.487024e-03	-77.023649	0.000000e+00	
college_degree_share	-3.722184e-02	5.818060e-03	-6.397638	1.608194e-10	
	2.5%	97.5%			
Coefficient					
Intercept	2.548957e-01	2.641989e-01			
exp_log_l1	3.472162e-02	4.480425e-02			
mon_valley	-1.787958e-03	3.189800e-03			
home_val	-3.796229e-08	-2.006365e-08			
pop_white_share	-1.964344e-01	-1.866850e-01			
college_degree_share	-4.862562e-02	-2.581805e-02			

Model 2 excluding interaction: Model 2 shows that closer proximity to industrial polluters increases vacancy rates in a neighborhood. Column 1 indicates that a 1 unit increase in the proximity index increases the vacancy rate by 4 percentage points, statistically significant at the 1 percent level. Being located in Mon Valley increases the vacancy rate by 0.1 percentage points. However, this result is not statistically significant. A 1 percent increase in the share of white population and college educated share decrease the vacancy rate by 19.2 and 3.7 percentage points respectively. The R^2 value of this model is 0.408.

```
[26]: fe_model4 = pf.feols("vacancy_rate ~ exp_log_l1 + mon_valley + home_val +
  ↪pop_white_share + college_degree_share + exp_log_l1:mon_valley",
  ↪data=model_data2, vcov="HC1")
fe_model4.coef()
fe_model4.se()
fe_model4.tidy()
```

```
[26]:
```

	Estimate	Std. Error	t value	Pr(> t)	\
Coefficient					
Intercept	2.588319e-01	2.352465e-03	110.025849	0.000000e+00	
exp_log_l1	8.050798e-02	3.902687e-03	20.628859	0.000000e+00	
mon_valley	1.944863e-02	1.369973e-03	14.196363	0.000000e+00	
home_val	-2.128828e-08	4.526902e-09	-4.702615	2.583323e-06	

pop_white_share	-1.962024e-01	2.484591e-03	-78.967708	0.000000e+00
college_degree_share	-3.877968e-02	5.742525e-03	-6.753071	1.481792e-11
exp_log_l1:mon_valley	-7.957463e-02	4.853374e-03	-16.395735	0.000000e+00

	2.5%	97.5%
Coefficient		
Intercept	2.542209e-01	2.634429e-01
exp_log_l1	7.285845e-02	8.815750e-02
mon_valley	1.676339e-02	2.213387e-02
home_val	-3.016131e-08	-1.241525e-08
pop_white_share	-2.010724e-01	-1.913325e-01
college_degree_share	-5.003541e-02	-2.752395e-02
exp_log_l1:mon_valley	-8.908757e-02	-7.006170e-02

Model 2 including interaction: Adding an interaction between distance to polluters and being located in Mon Valley increases the strength of our model. In column 2, an 1 unit increase in the proximity index increases the vacancy rate by 8.1 percentage points for neighborhoods not in Mon Valley. However, if a neighborhood is in Mon Valley, the effect on vacancy rates is only an increase of 0.1 percentage points. The R^2 value of this model is slightly strong than column 1, at 0.416.

```
[27]: # pf.etable(
#       [fe_model3, fe_model4],
#       labels={
#           "exp_log_l1": "(Weighted) Proximity Index (exp_log_l1)",
#           "mon_valley": "Mon Valley Dummy",
#           "home_val": "Median Home Value",
#           "pop_white_share": "Share of White Population",
#           "college_degree_share": "Share of Population with College Degree",
#           "exp_log_l1:mon_valley": "exp_log_l1 x Mon Valley Dummy"
#       },
#       caption="Vacancy Rate: Naive OLS vs Interaction"
# )
```

1.7 Conclusions, Limitations, and Areas for Further Study

We find that being located near a polluter decreases residential property sale prices and increases neighborhood vacancy rates. Therefore, we can reject both of our null hypotheses. However, we are surprised to find that being located in Mon Valley decreases the effect of living near a polluters on both residential property sale prices and neighborhood vacancy rates. This is especially surprising given that the average sale prices of residential properties, and market values of the buildings and land in Mon Valley are noticeably lower than for residential properties outside of Mon Valley. There may be some characteristics of

One way to read regression results is that the steel facilities have a positive impact on nearby property values, presumably because of the economic activity and employment opportunities they generate in the surrounding area. However, [recent research on the employment characteristics of these facilities by the Carnegie Mellon CREATE Lab](#) suggests that the jobs they provide may not be as accessible or beneficial to the local population as one might assume, potentially limiting the

positive impact on property values; as such we caution against this interpretation.

1.7.1 Limitations:

Our measure of local pollution measures straight-line distance, not air quality. It also ignores wind direction and the river valleys, both of which have a [substantial impact](#) on where emissions actually go in the Mon Valley. Additionally, TRI emissions are self-reported and cover one year, and TRI doesn't track fine particulate matter, sulfur dioxide, or nitrogen oxides, which are a large share of the Clairton coke works' impact. We picked facilities using 2022 RSEI scores and 2024 TRI pounds, so a facility whose emissions changed between those years could be misclassified. TMS International and Edgar Thomson are reported about 30 feet apart in Braddock, so the unweighted index counts that site twice. Additionally, lacking easily accessible building footprint data, each facility is represented as a single point, which understates proximity for parcels next to large sites like Clairton, which is almost 400 acres, or half a mile. It is impossible to say whether the index as we created it realistically represents the extent to which pollution is felt by those living adjacent to Mon Valley steel factories, compared to the smaller facilities also included.

It is also possible that emissions from preceding decades had an impact on property values in ways that are not obvious; several steel factories south of Pittsburgh have closed since the late 20th century, such as the Homestead Works and the Duquesne Works, which closed in the 1980s and 1990s, respectively. While the facilities are no longer operational, their historical emissions may still have lingering effects on the surrounding environment and property values. A panel data analysis would be able to better account for the historical impact of emissions on property values by tracking changes over time and controlling for unobserved factors that may influence both pollution and housing prices.

In the regression analysis, vacancy rate appears to have a significant impact on property values because of extreme right skew in its distribution. This skewness disproportionately influences the regression results, making it appear more important than it might be in a more normally distributed context, and in turn underrating the impact of other variables.

Other factors impacting housing value that we did not include in our analysis because of scope include school quality, neighborhood crime rates, proximity to amenities such as job centers, parks, shopping centers, and transportation, and broader economic conditions like employment rates and income levels.

1.7.2 References

American Lung Association. (2026, April 22). Most Polluted Places to Live | State of the Air. <https://www.lung.org/research/sota/key-findings/most-polluted-places>

Allegheny County Office of Property Assessments. (2026). *Allegheny County property assessments* [Data set]. Western Pennsylvania Regional Data Center. Retrieved October 2026, from <https://data.wprdc.org/dataset/property-assessments>

Allegheny County. (2026). *Allegheny County parcel boundaries* [Data set]. Western Pennsylvania Regional Data Center. Retrieved October 2026, from <https://data.wprdc.org/dataset/allegheny-county-parcel-boundaries1>

CREATE Lab. (2025, November). *Employment characteristics of steel facilities*

in the Mon Valley. Carnegie Mellon University. https://d8310d7c-75ad-4dd2-92cd-2709a344c990.filesusr.com/ugd/5c8c0e_d899b7325a374035ba505c470b7f2473.pdf

Currie, J., Davis, L., Greenstone, M., & Walker, R. (2015). Environmental health risks and housing values: Evidence from 1,600 toxic plant openings and closings. *American Economic Review*, 105(2), 678–709.

Environment America Research & Policy Center. (2023). *Allegheny County's toxic ten*. <https://environmentamerica.org/pennsylvania/center/resources/allegheny-countys-toxic-ten/>

U.S. Environmental Protection Agency. (n.d.). *EasyRSEI dashboard* (RSEI Version 2.3.12). Retrieved October, 2026, from <https://edap.epa.gov/public/extensions/EasyRSEI/EasyRSEI.html>

U.S. Environmental Protection Agency. (n.d.). *TRI basic data files: Calendar years 1987–present* [Data set]. Toxics Release Inventory Program. Retrieved October 2026, from <https://www.epa.gov/toxics-release-inventory-tri-program/tri-basic-data-files-calendar-years-1987-present>.

U.S. Census Bureau. (2026). *2020–2024 American Community Survey 5-year estimates*. Retrieved October 2026, from `pytidycensus` package.

Wiggin, J. (2023). [Is Pittsburgh's weird terrain making the already-dodgy air quality even dodgier?](#). *Pittsburgh City Paper*.